UTM
UNIVERSITI TEKNOLOGI MALAYSIA



Getting Started with R Programming

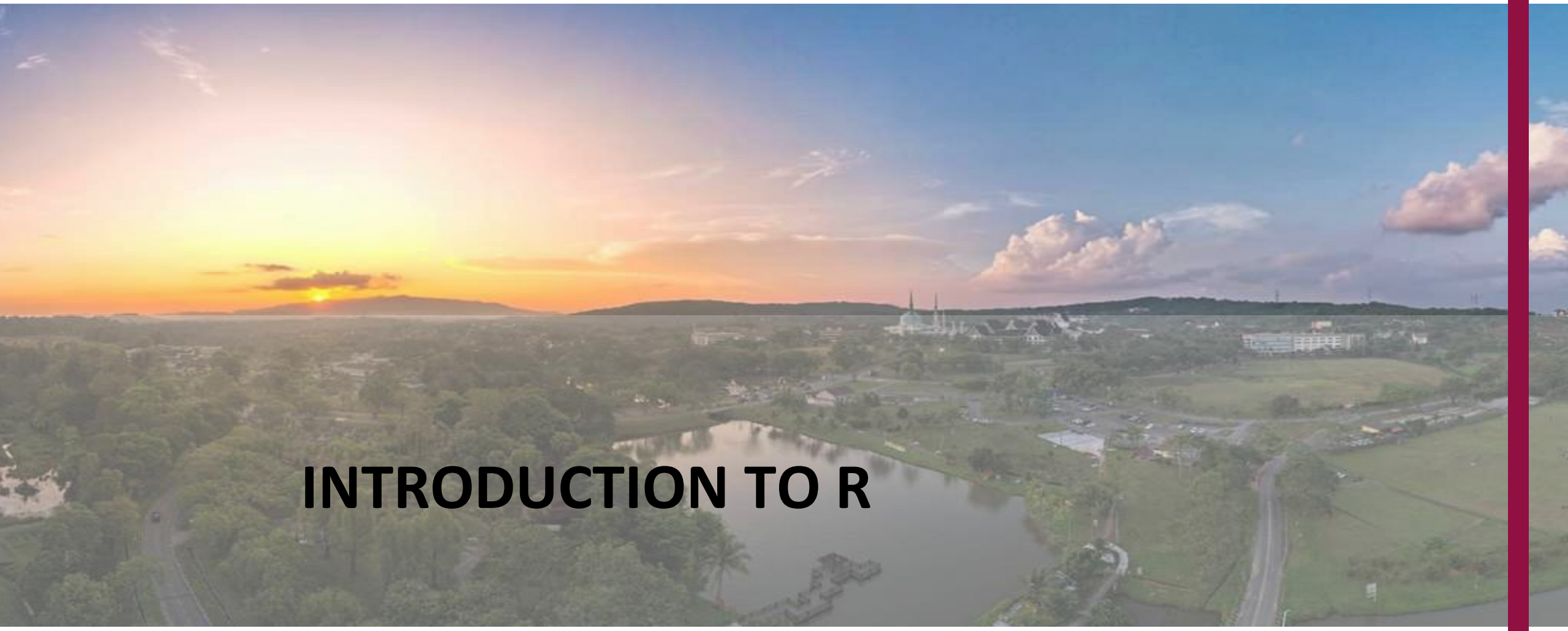
Innovating Sustainable Solutions

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Outline

- Introduction to R
- R Basics
- Packages in R
- Search Help in R
- Functions in R
- Control Structures in R
- Import/Export in R
- Statistics with R

UTM
UNIVERSITI TEKNOLOGI MALAYSIA



INTRODUCTION TO R

Innovating Sustainable Solutions

R……a mythical beast?

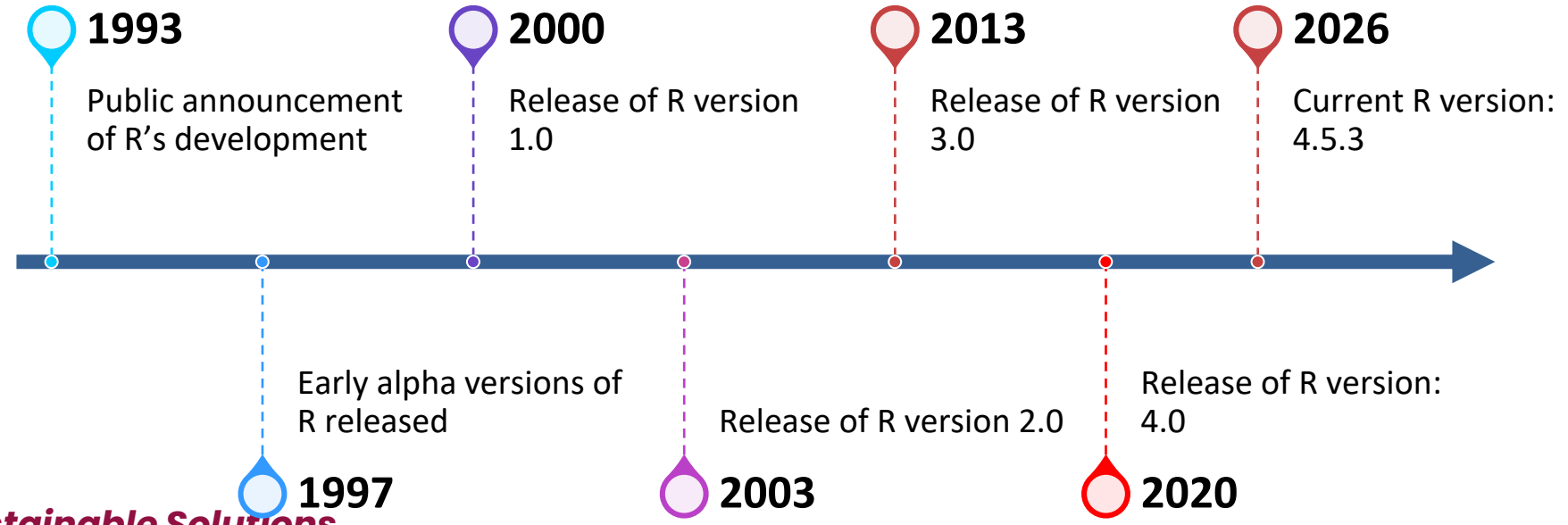


- R is **free** and **open-source**
- R is a **command-line** programming language
- It is not a statistics program! But a programming language that **works well for statistics**
- With **visualization capabilities** and **highly extensible**.
- Operating system independent (supported in: Windows / macOS and Linux)
- R can be integrated with other packages and languages (Excel, SAS, SPSS, etc.)

A Brief History



- R was developed by **Ross Ihaka** and **Robert Gentleman**
- The original goal was to develop software for students to use
- They were encouraged to make it an open-source project.



Using R

- **Two** ways:
  1. Through R environment setup in your local computer (requires installation of R environment and RStudio (optional)). **OR**
  2. Through RStudio Cloud (https://rstudio.cloud/)

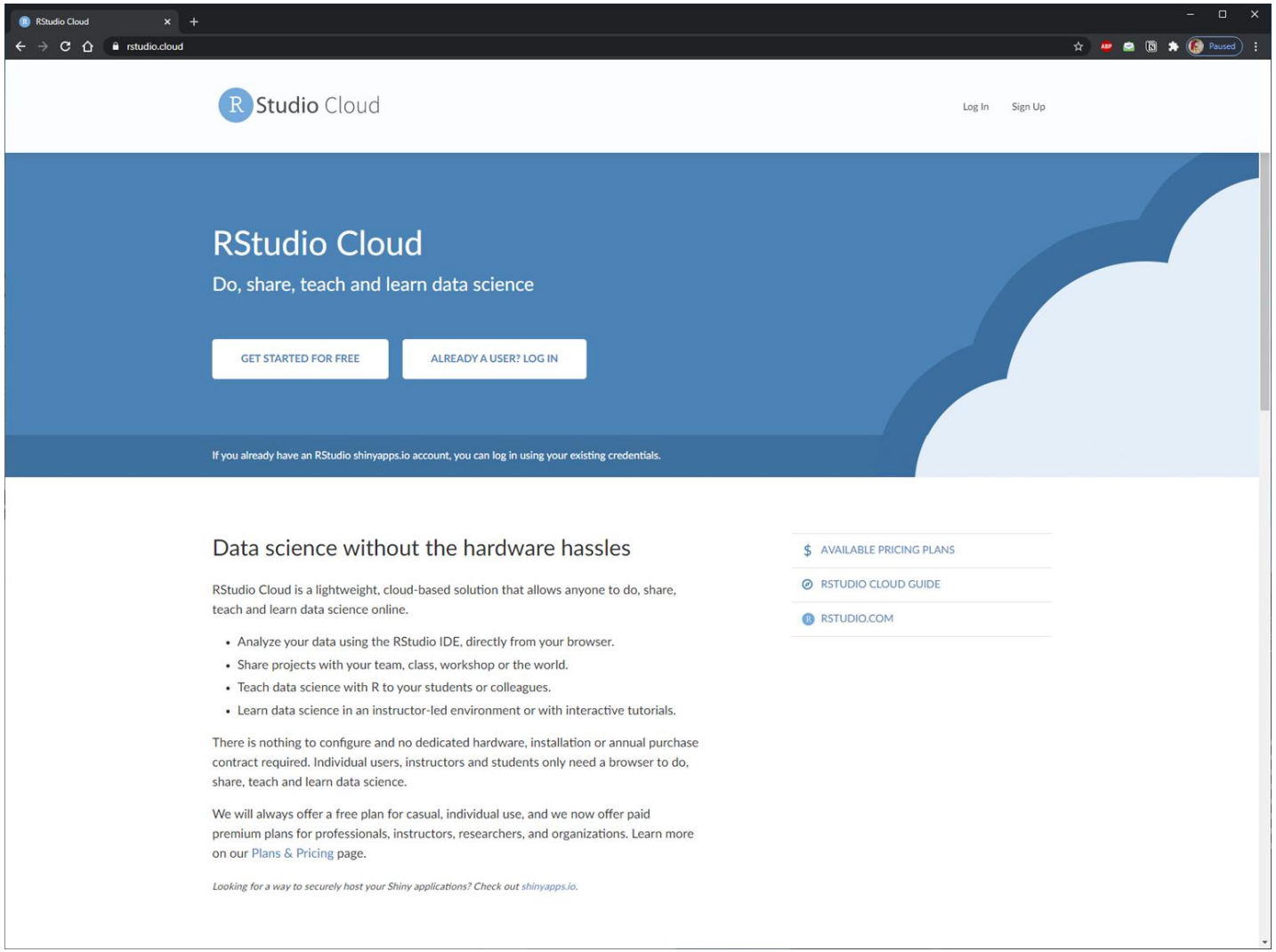




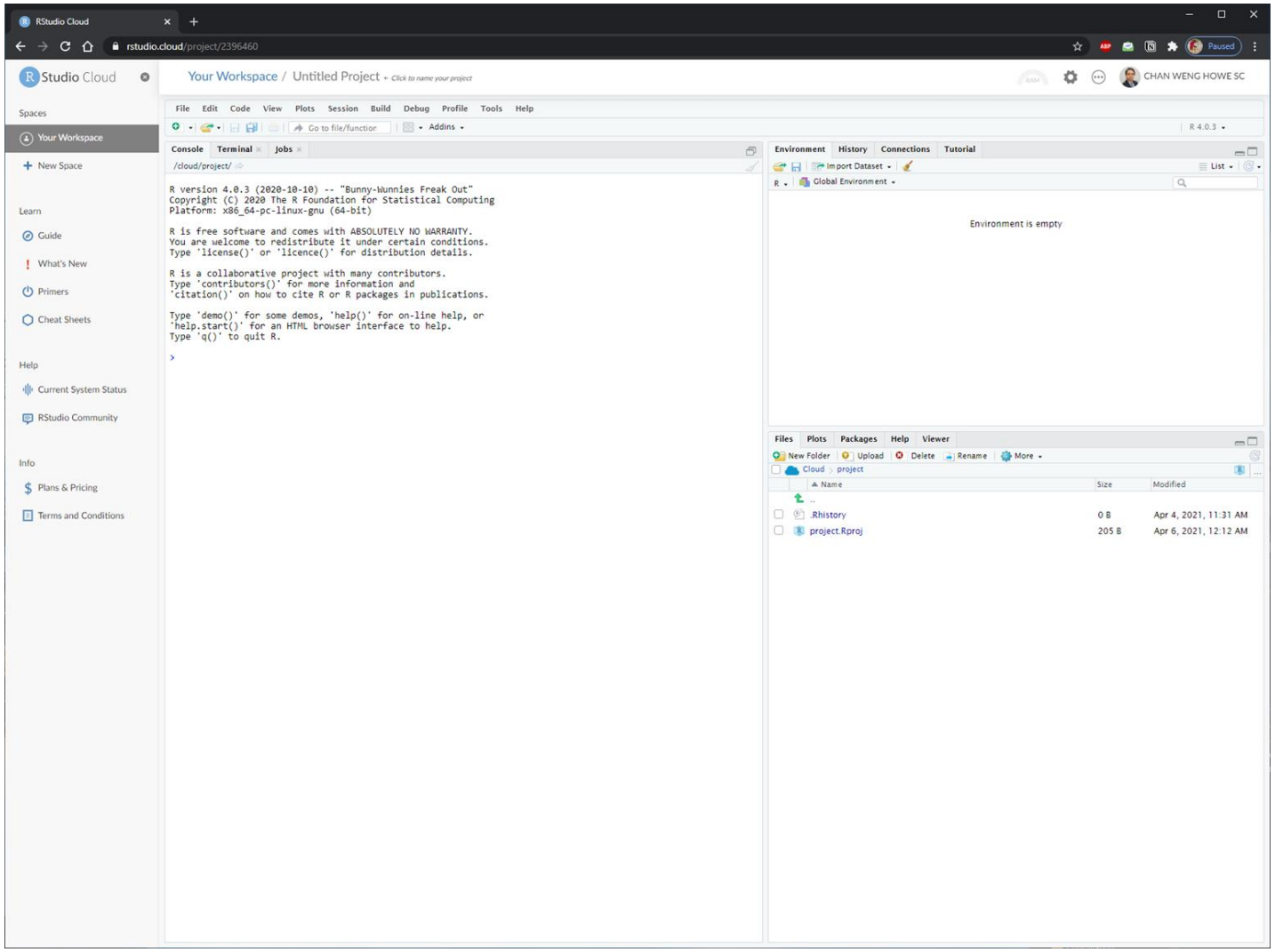


UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Using RStudio Cloud



Using RStudio Cloud



Setting up R in local computer

- 2 components:
  - R Programming environment (https://www.r-project.org/)
  - RStudio IDE for R (https://www.rstudio.com/products/rstudio)
- R Programming environment provides the **core files** and **libraries** of R
- RStudio act as the Integrated Development Environment (IDE) that provides an **additional layer of GUI** which is consistent across Windows, macOS, Linux and RStudio Cloud.

Installation of R 4.5.3

- R 4.5.3 can be downloaded at:
  - https://cran.r-project.org/bin/macosx/ (macOS)
  - https://cran.r-project.org/bin/windows/base/ (Windows)
- This should be installed before the installation of RStudio.

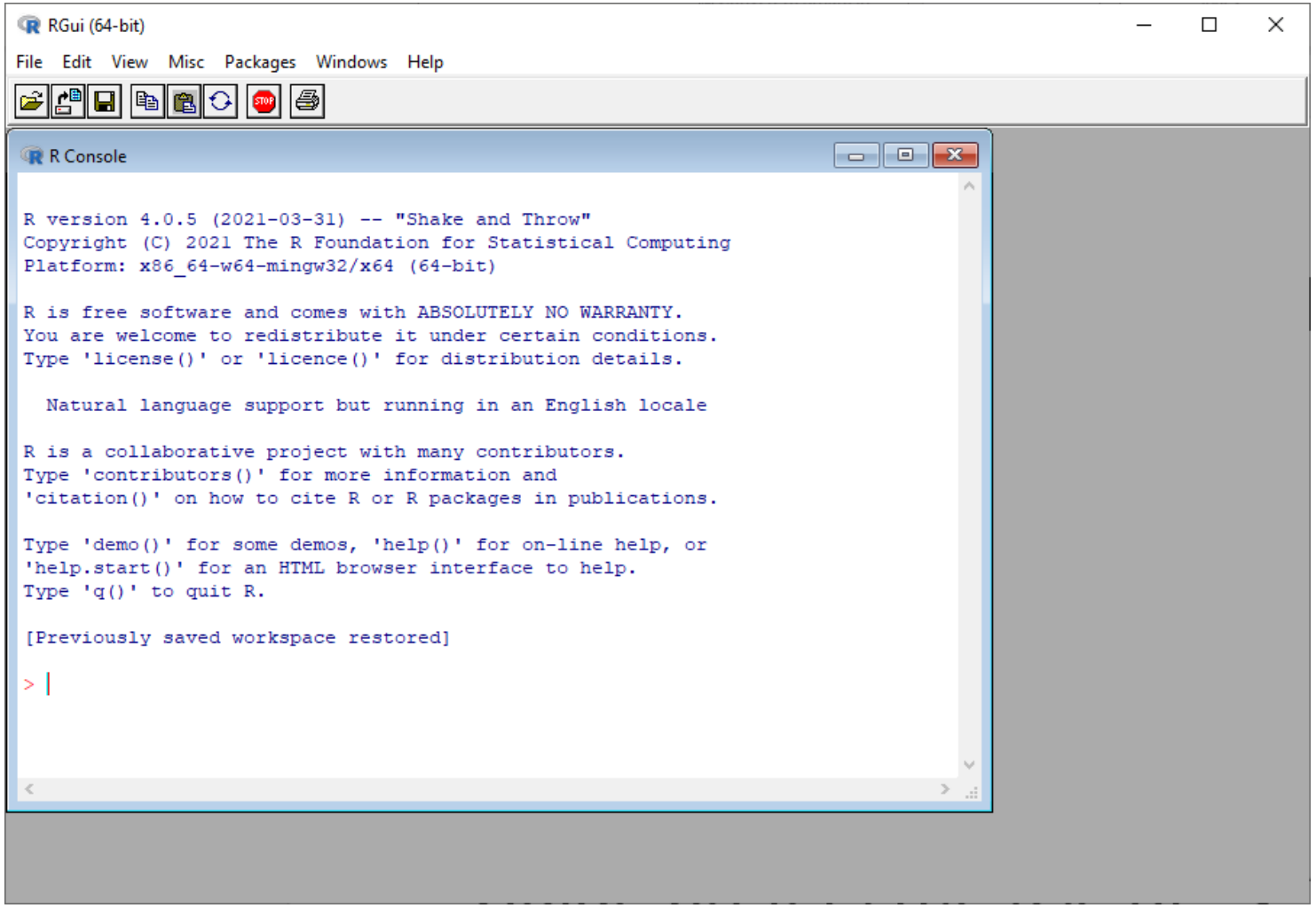
Installation of Rstudio IDE

- RStudio can be downloaded at:
  - https://download1.rstudio.org/desktop/macos/RStudio-1.4.1106.dmg (macOS)
  - https://download1.rstudio.org/desktop/windows/RStudio-1.4.1106.exe (Windows)

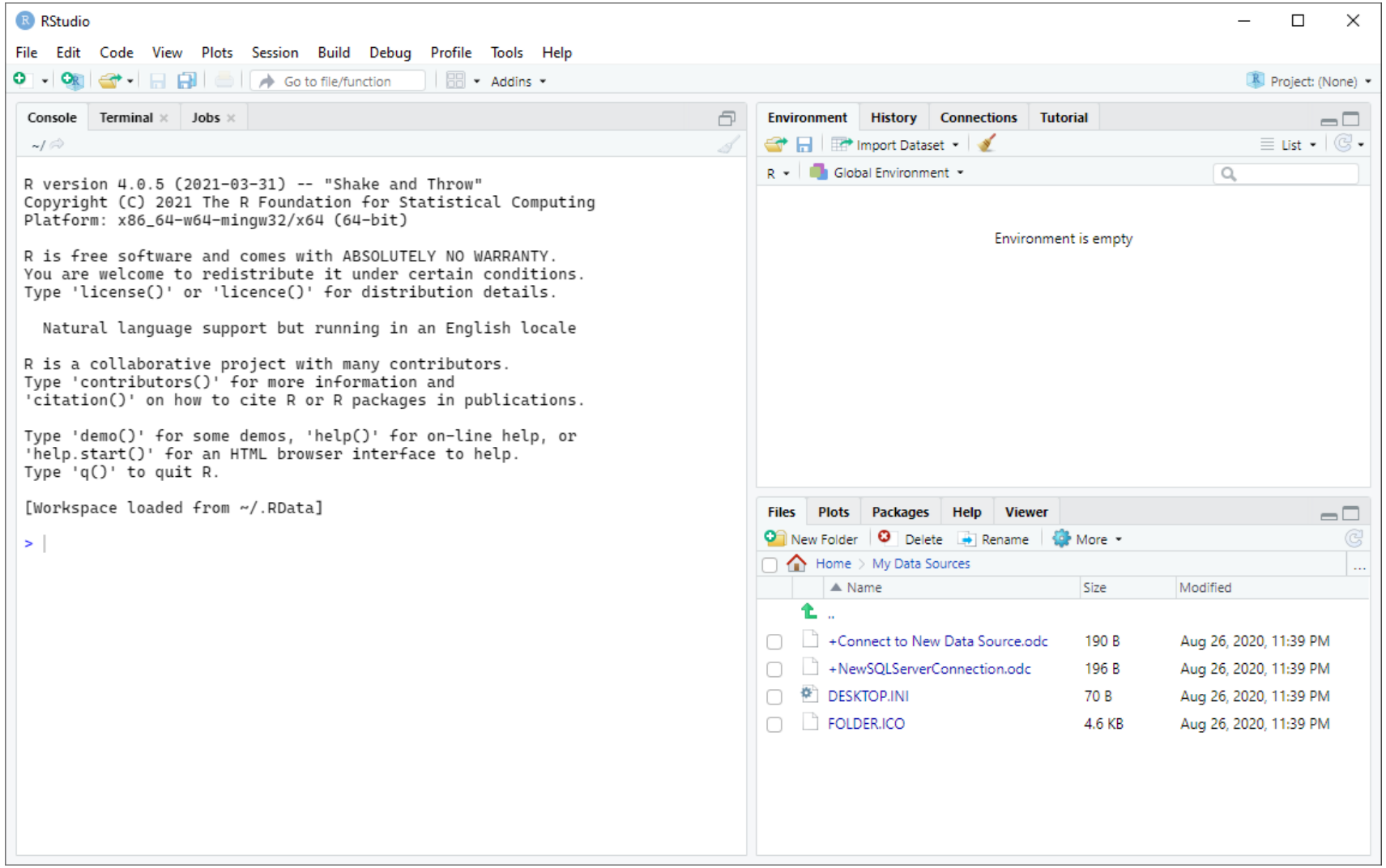
Accessing R in local computer

- 3 Ways to access R in Windows / macOS:
  i. via **R Console** (come together with R environment)
  ii. via **Command Prompt** (Win) / **Terminal** (macOS)
    - Type “R” in the command prompt or terminal and press enter to initialize the R environment.
  iii. via **RStudio**
    - A more organized and integrated UI

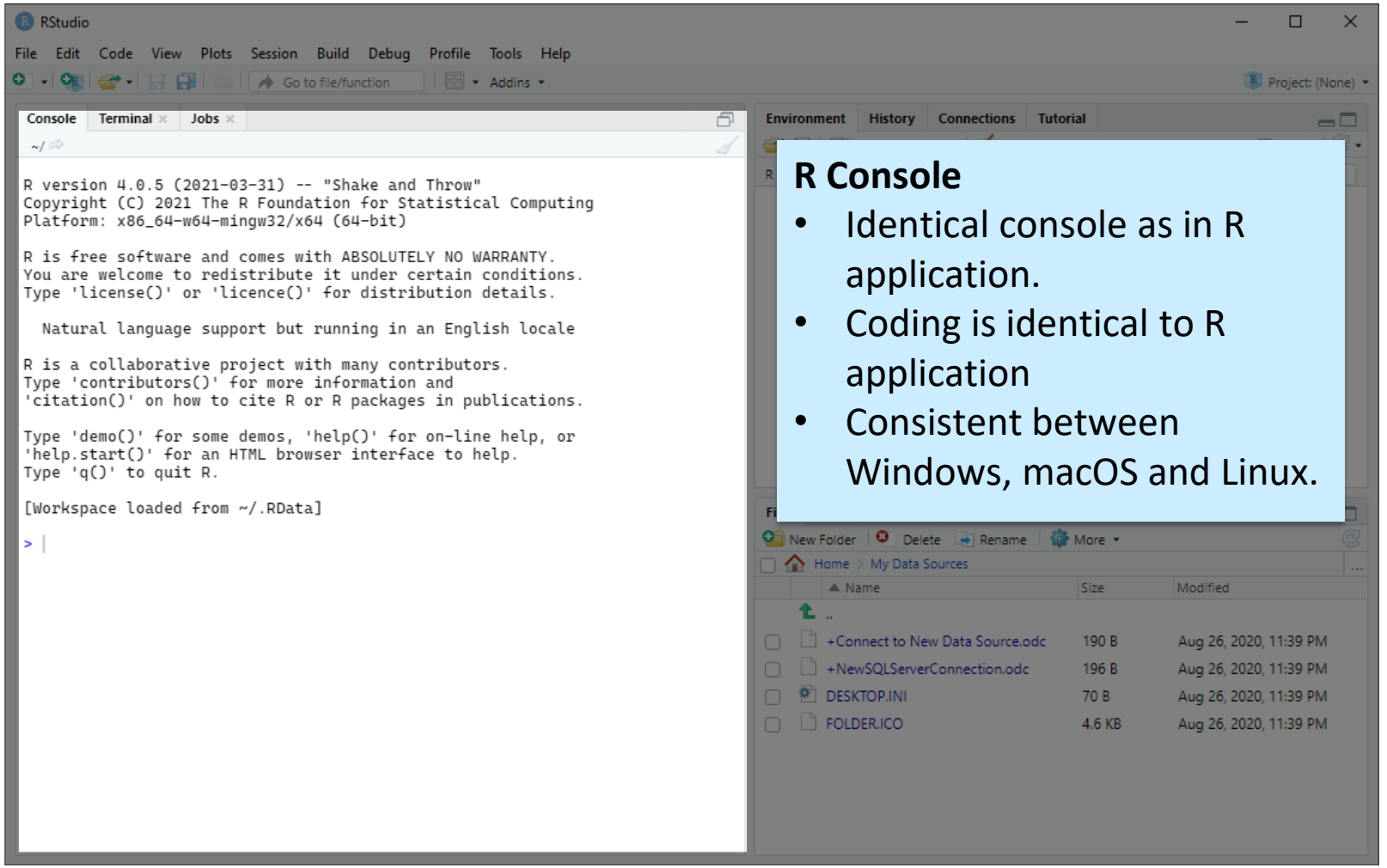
R Console Interface



RStudio Interface



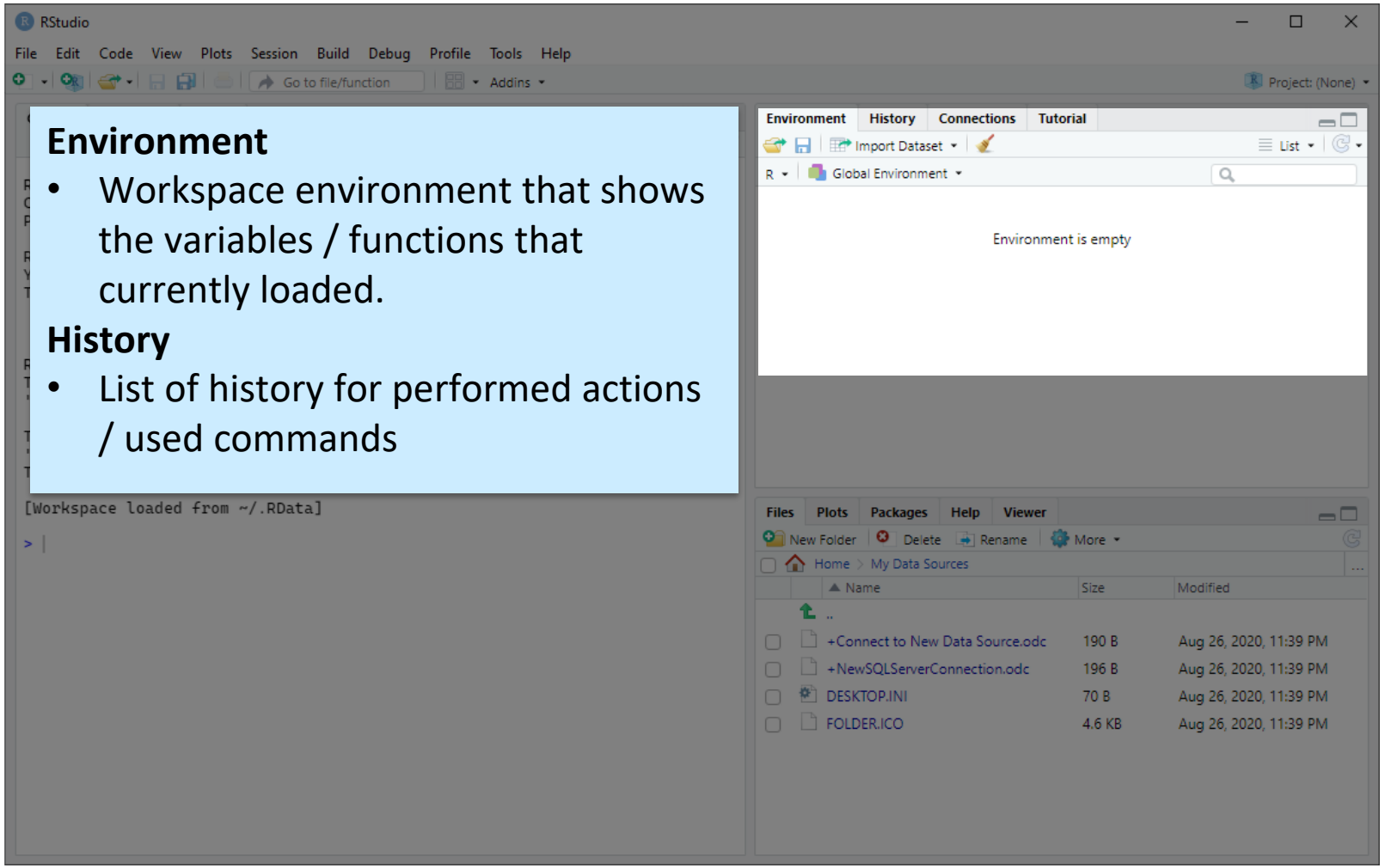
RStudio Interface



R Console

- Identical console as in R application.
- Coding is identical to R application
- Consistent between Windows, macOS and Linux.

RStudio Interface



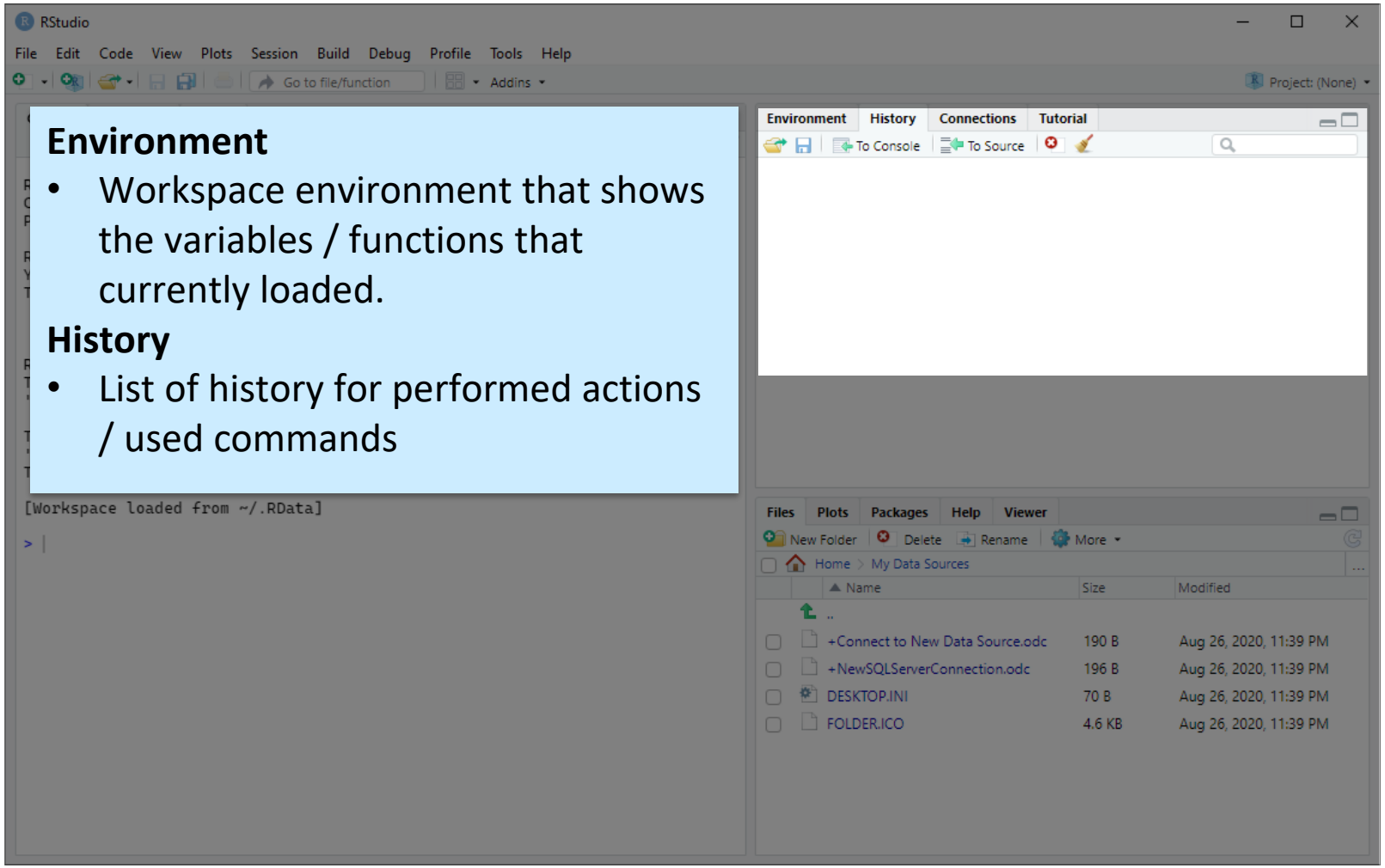
Environment

- Workspace environment that shows the variables / functions that currently loaded.

History

- List of history for performed actions / used commands

RStudio Interface



Environment

- Workspace environment that shows the variables / functions that currently loaded.

History

- List of history for performed actions / used commands

RStudio Interface

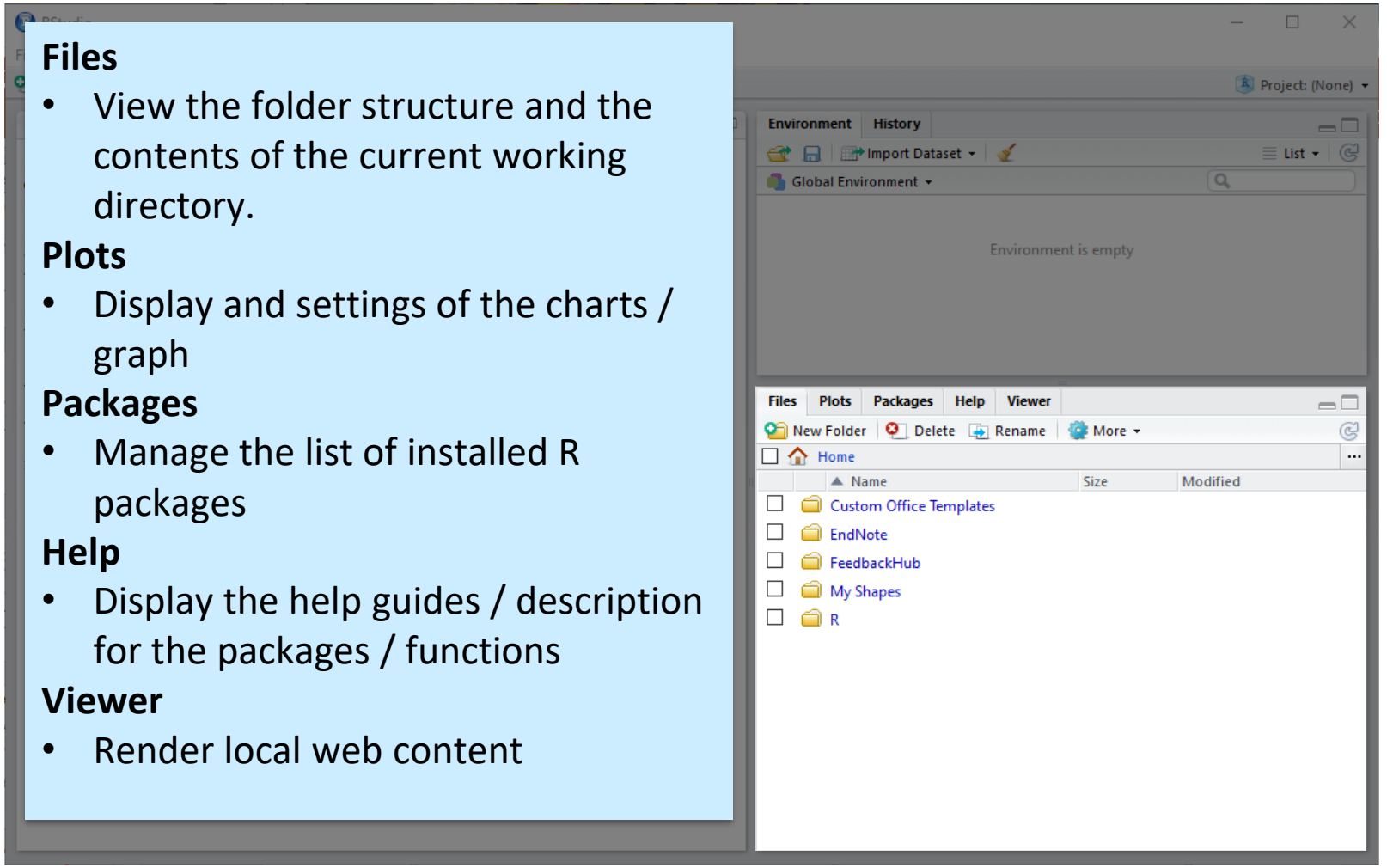
Files
- View the folder structure and the contents of the current working directory.

Plots
- Display and settings of the charts / graph

Packages
- Manage the list of installed R packages

Help
- Display the help guides / description for the packages / functions

Viewer
- Render local web content



RStudio Interface

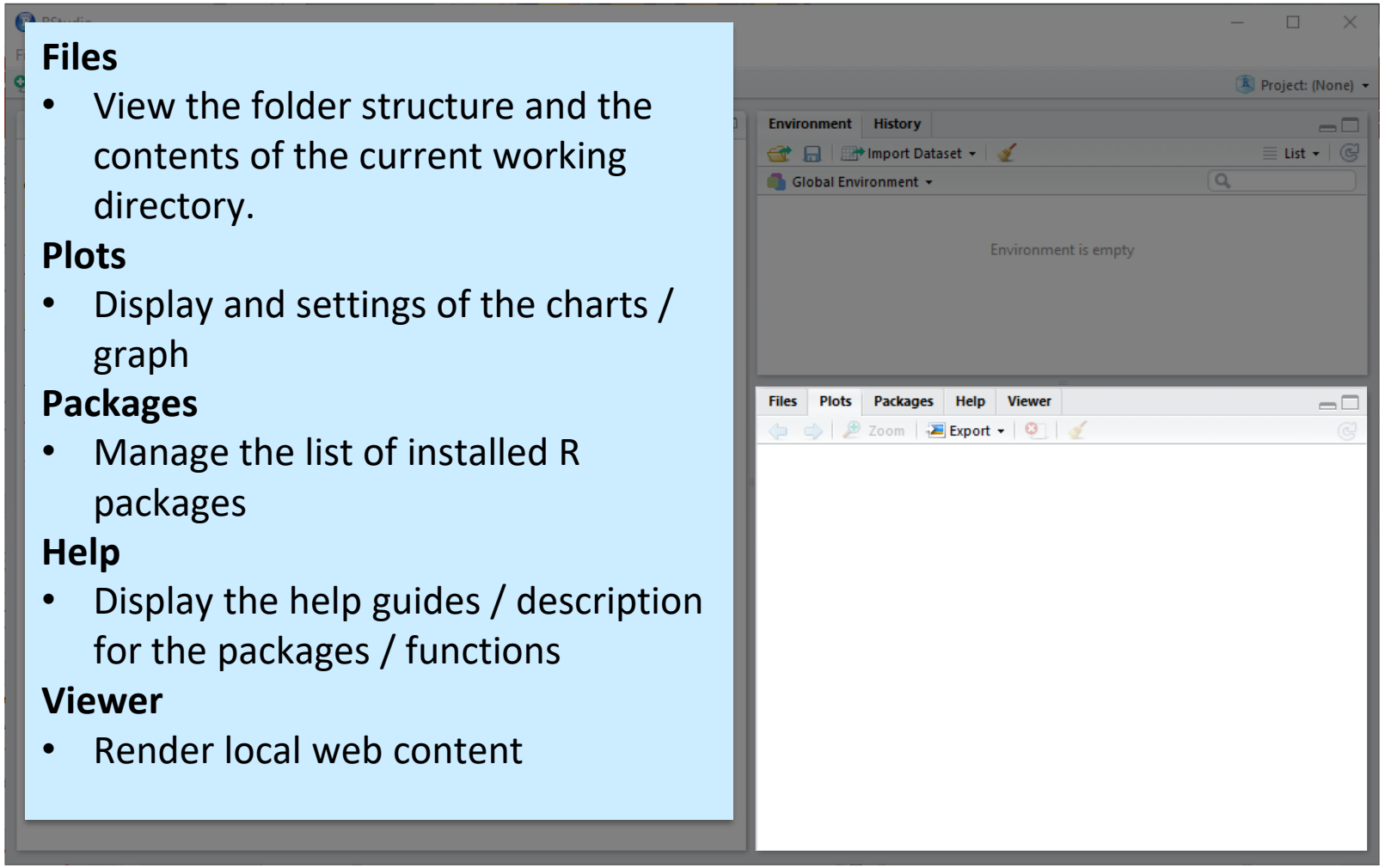
Files
- View the folder structure and the contents of the current working directory.

Plots
- Display and settings of the charts / graph

Packages
- Manage the list of installed R packages

Help
- Display the help guides / description for the packages / functions

Viewer
- Render local web content



RStudio Interface

Files

- View the folder structure and the contents of the current working directory.

Plots

- Display and settings of the charts / graph

Packages

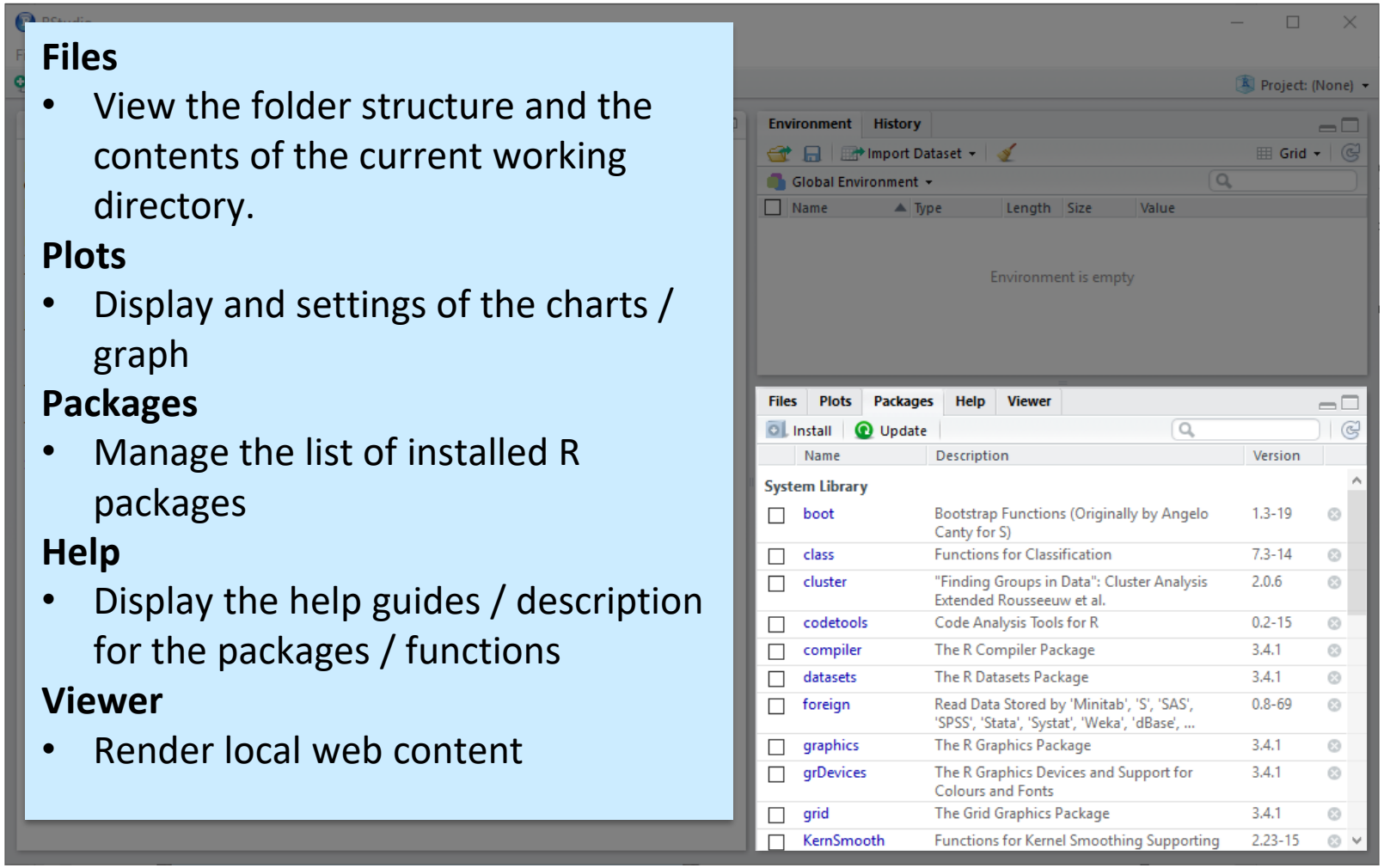
- Manage the list of installed R packages

Help

- Display the help guides / description for the packages / functions

Viewer

- Render local web content



RStudio Interface

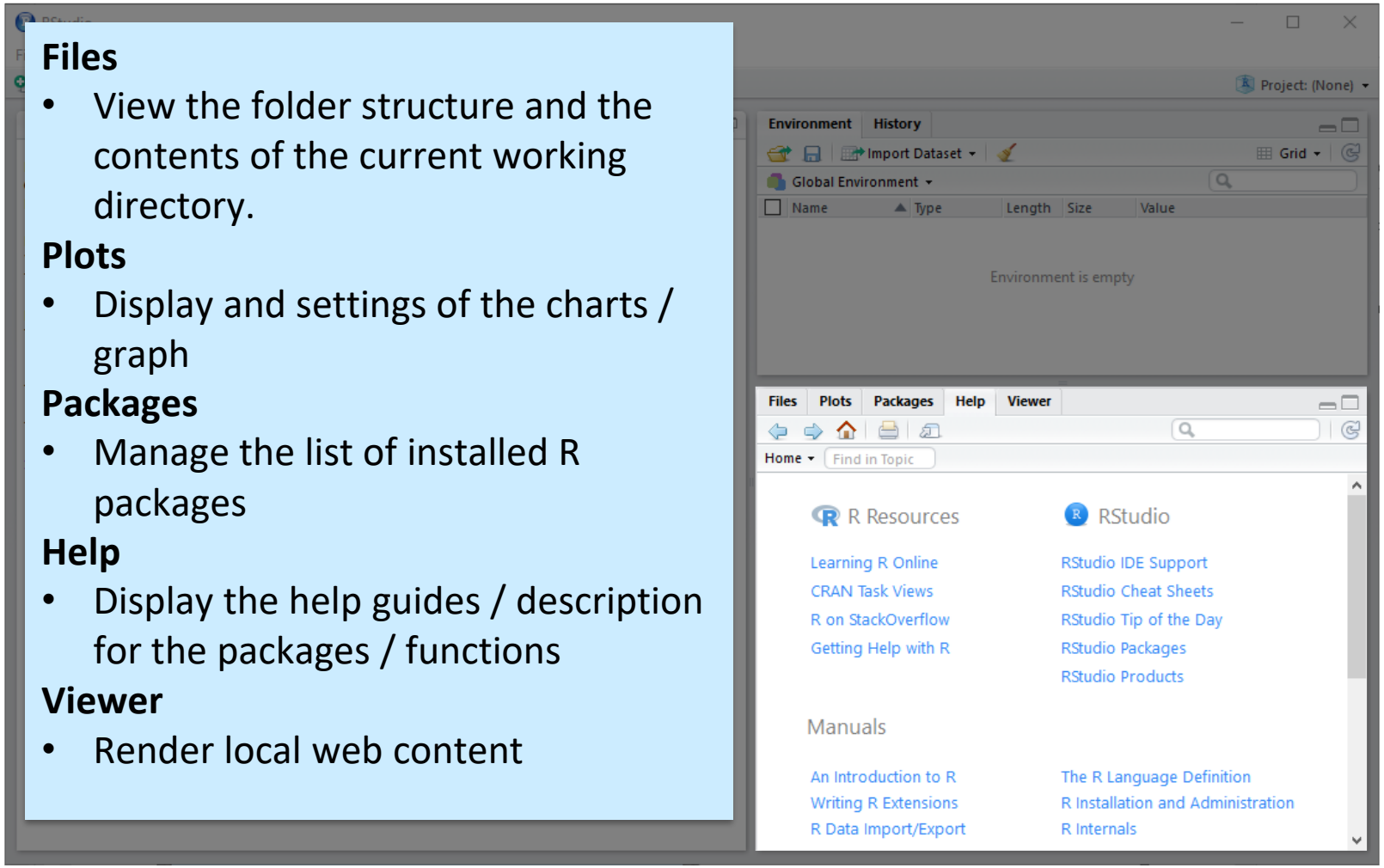
Files
- View the folder structure and the contents of the current working directory.

Plots
- Display and settings of the charts / graph

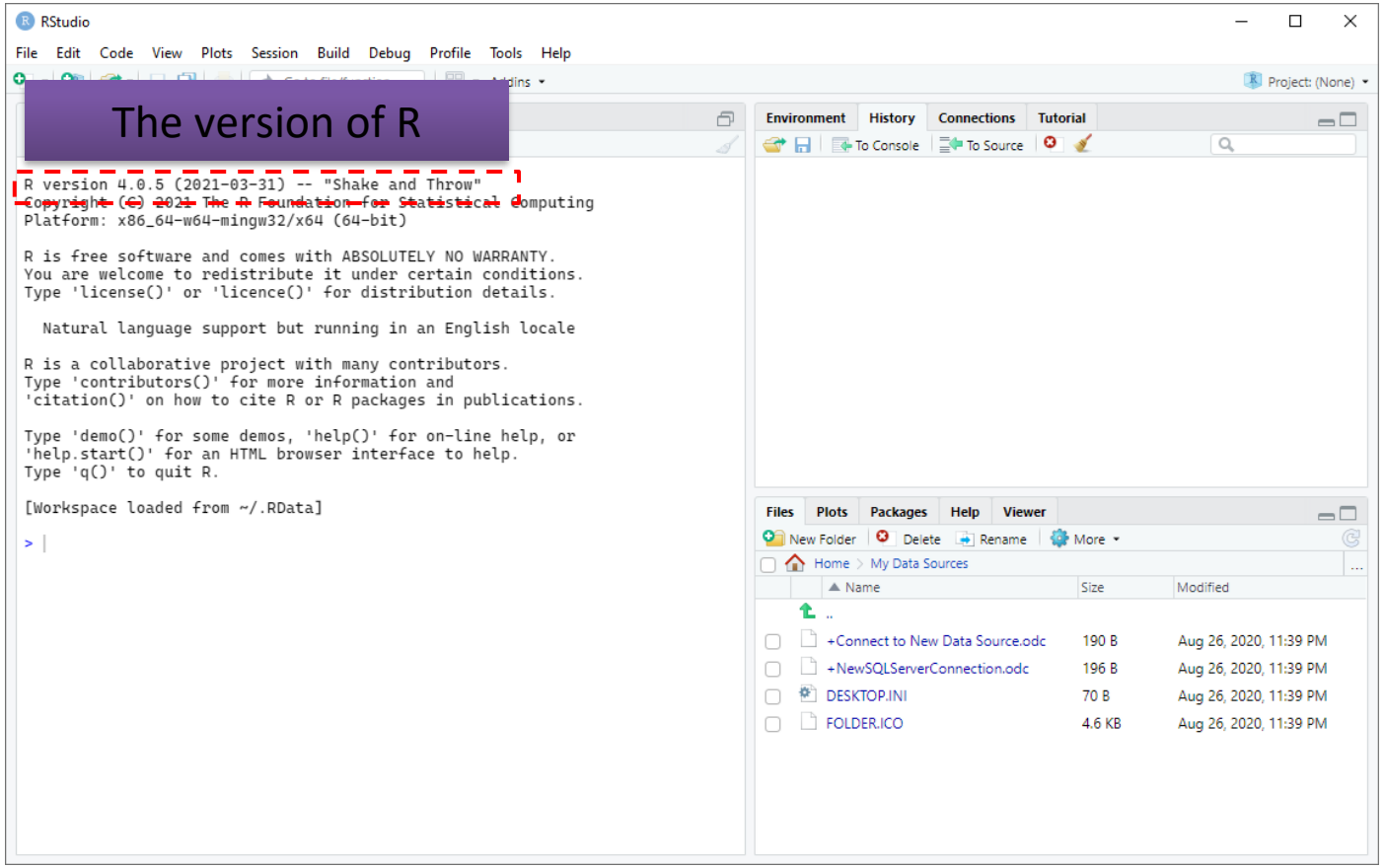
Packages
- Manage the list of installed R packages

Help
- Display the help guides / description for the packages / functions

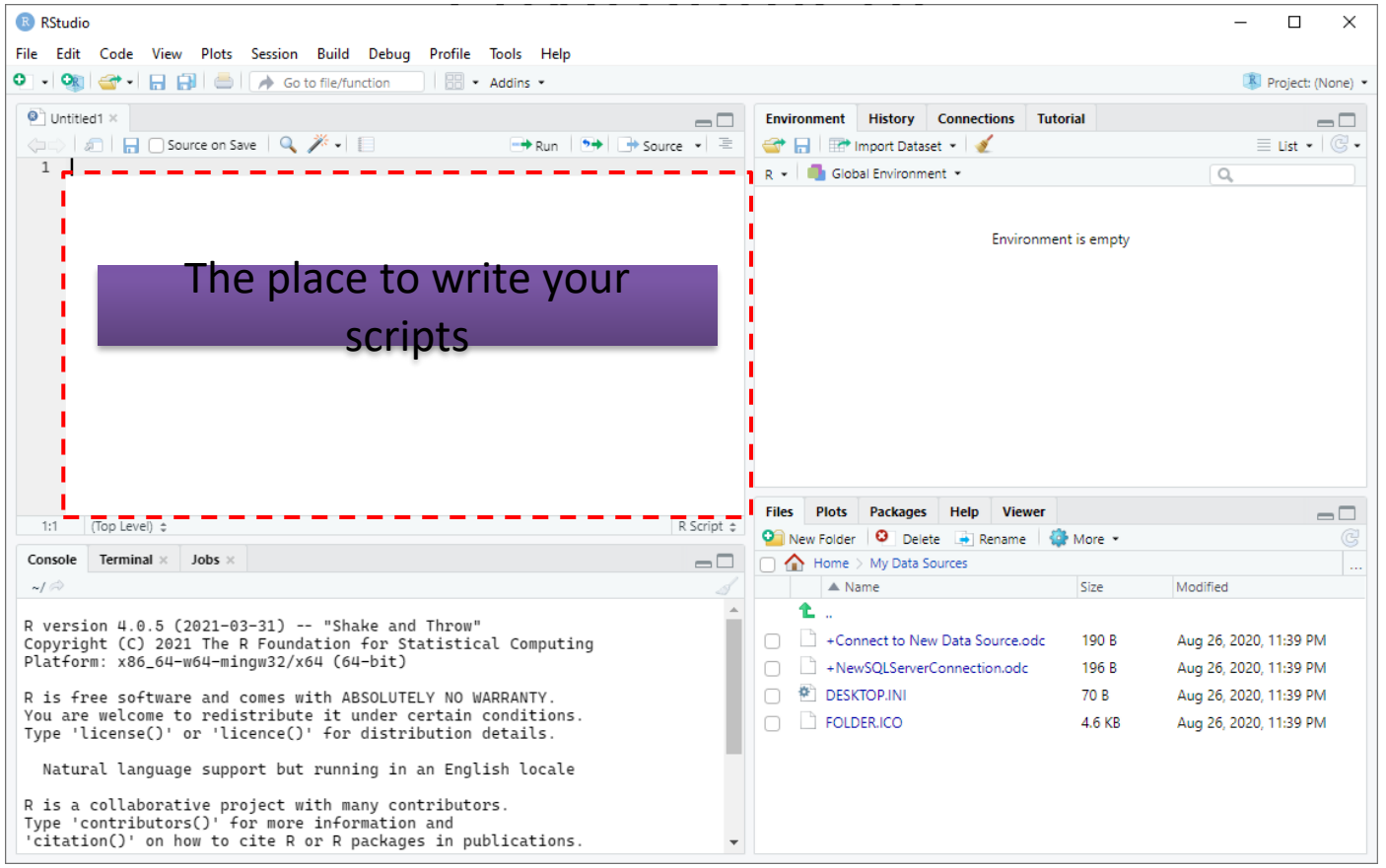
Viewer
- Render local web content



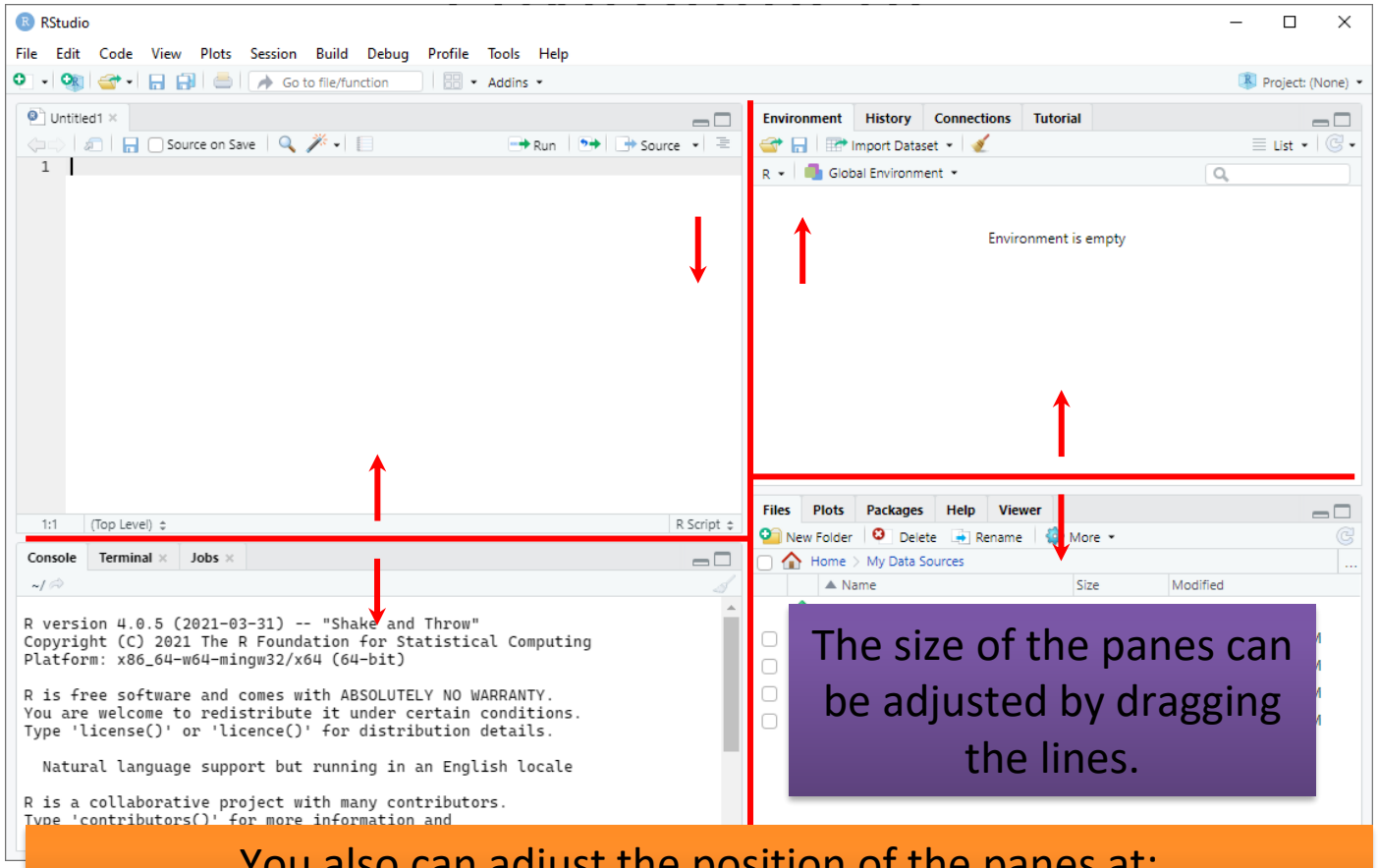
Getting Started with the R Environment



Getting Started with the R Environment

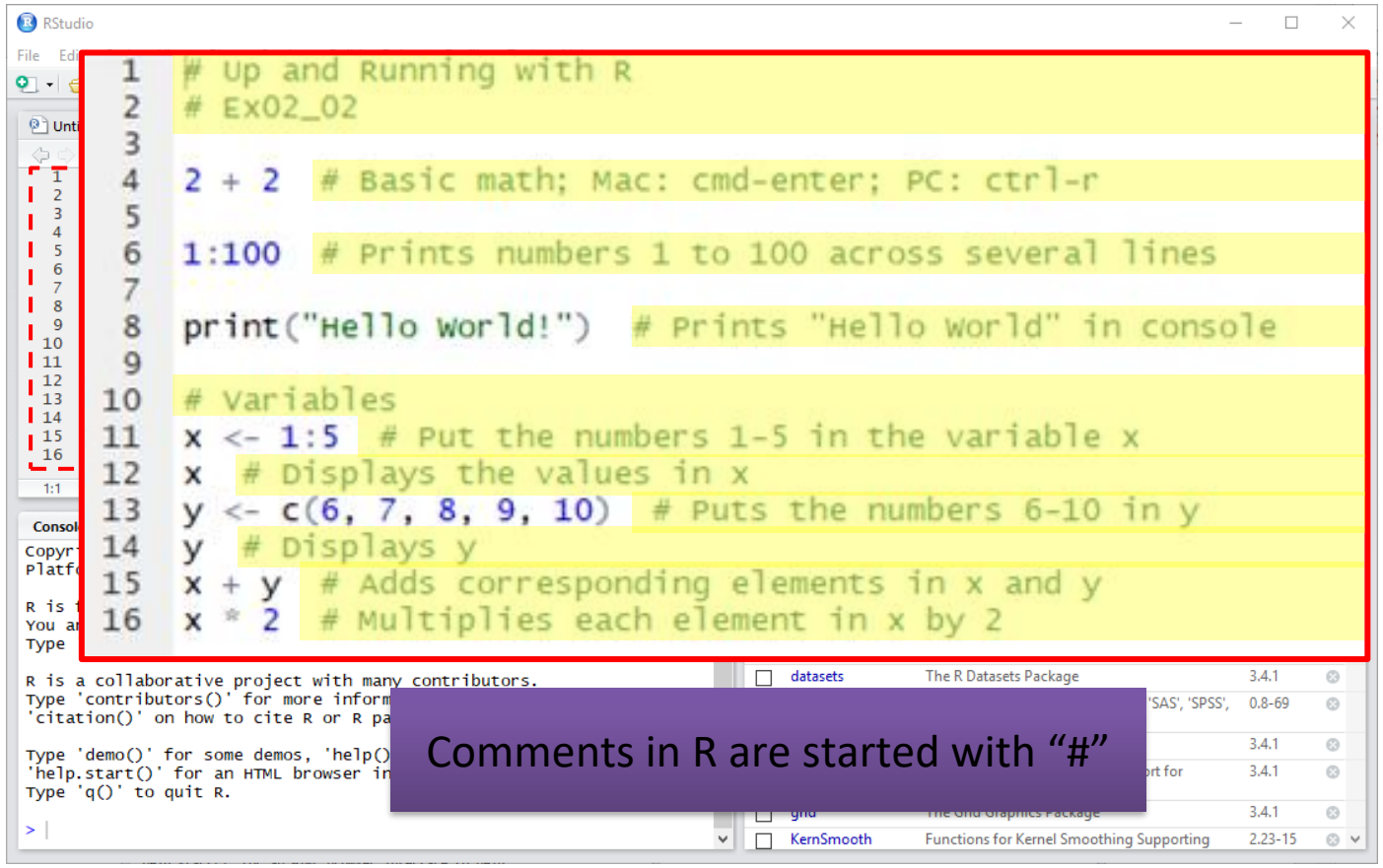


Getting Started with the R Environment

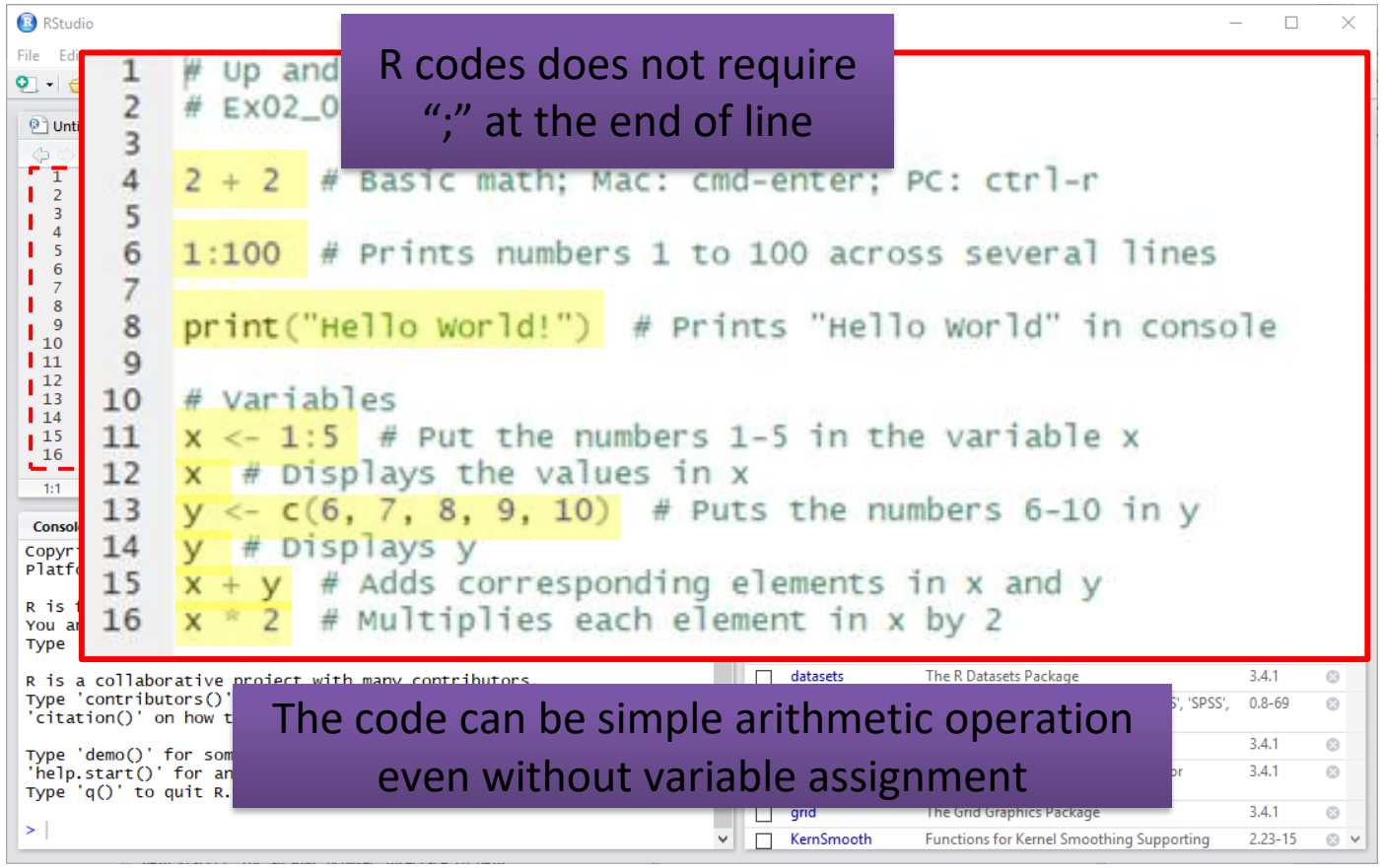


You also can adjust the position of the panes at:
Tools > Global Options > Pane Layout

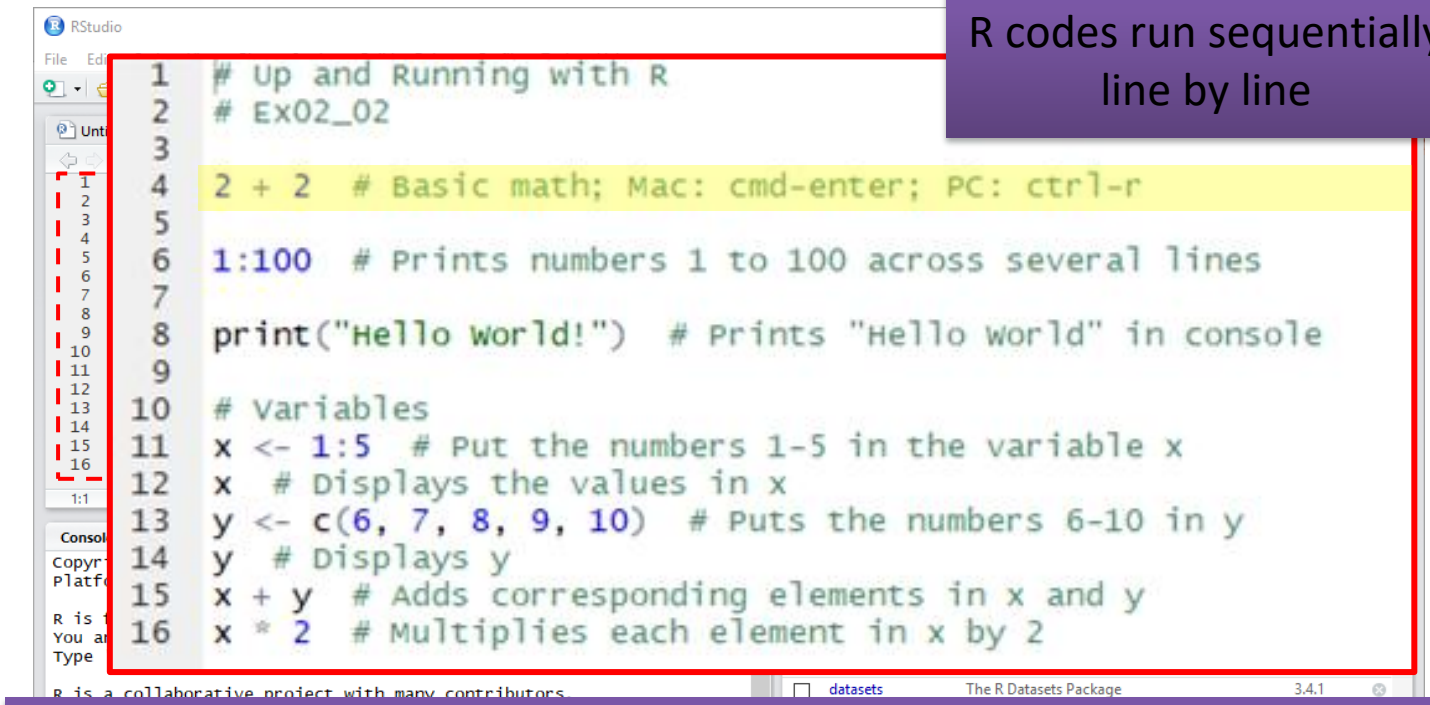
Getting Started with the R Environment



Getting Started with the R Environment



Getting Started with the R Environment



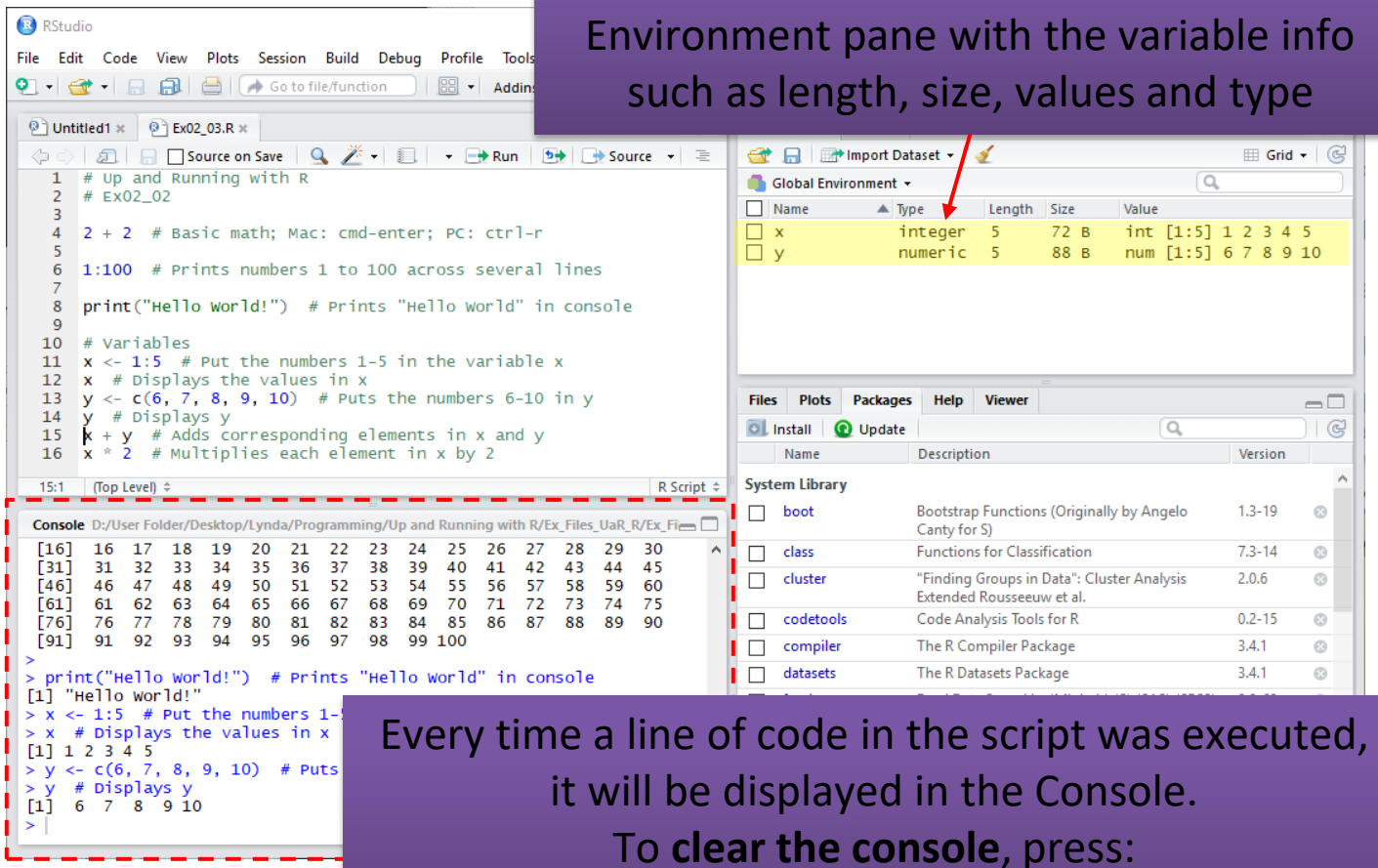
R codes run sequentially line by line

You can run a single line of code by putting a cursor at the start of the line, or run specific part of code by highlighting the code,
Then, press:

Command + **Enter** (macOS) / **CTRL** + **Enter** (Windows)

Getting Started with the R Environment

Variables created will be listed in the Environment pane with the variable info such as length, size, values and type

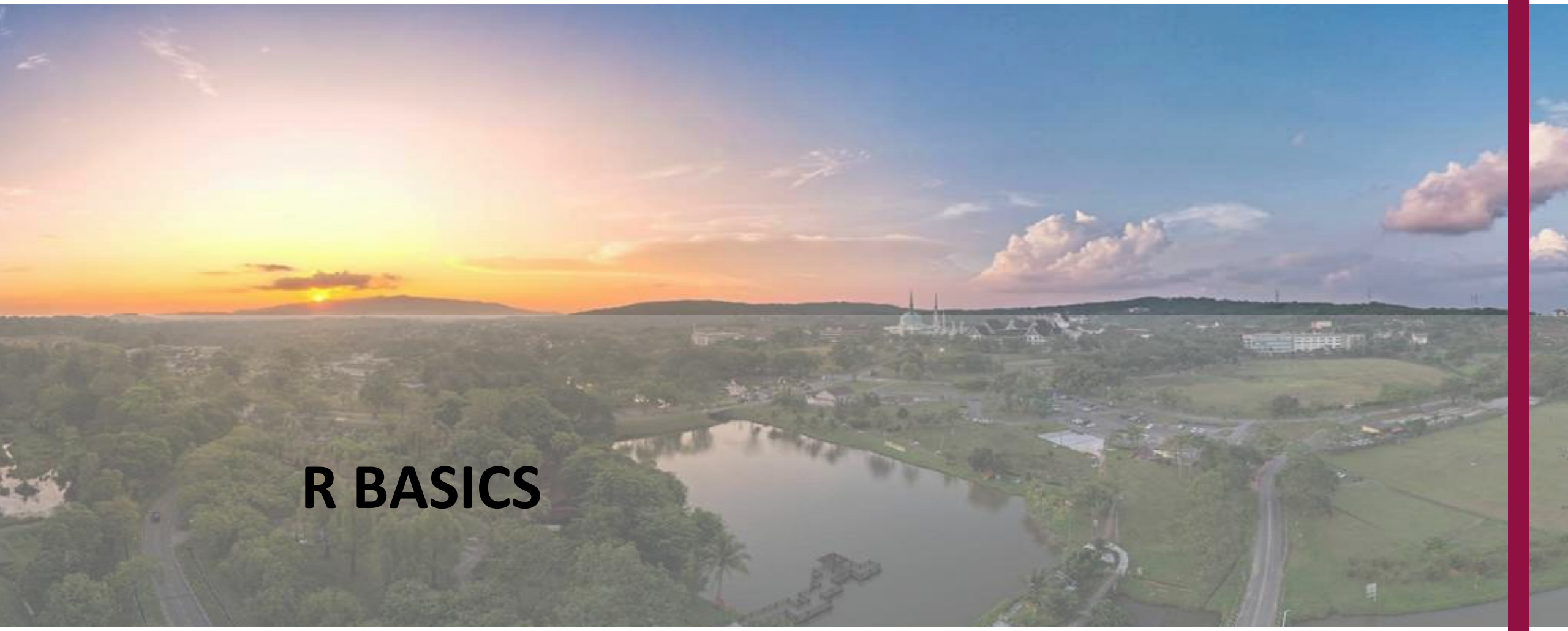


Every time a line of code in the script was executed, it will be displayed in the Console.

To **clear the console**, press:

Command + **L** (macOS) / **CTRL** + **L** (Windows)

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

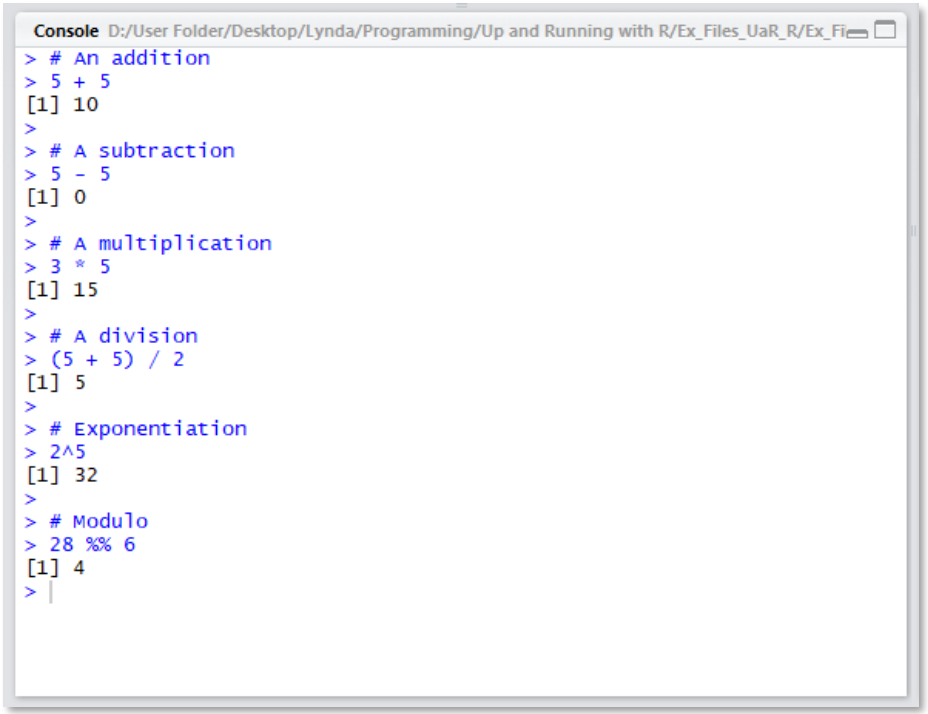


R BASICS

Innovating Sustainable Solutions

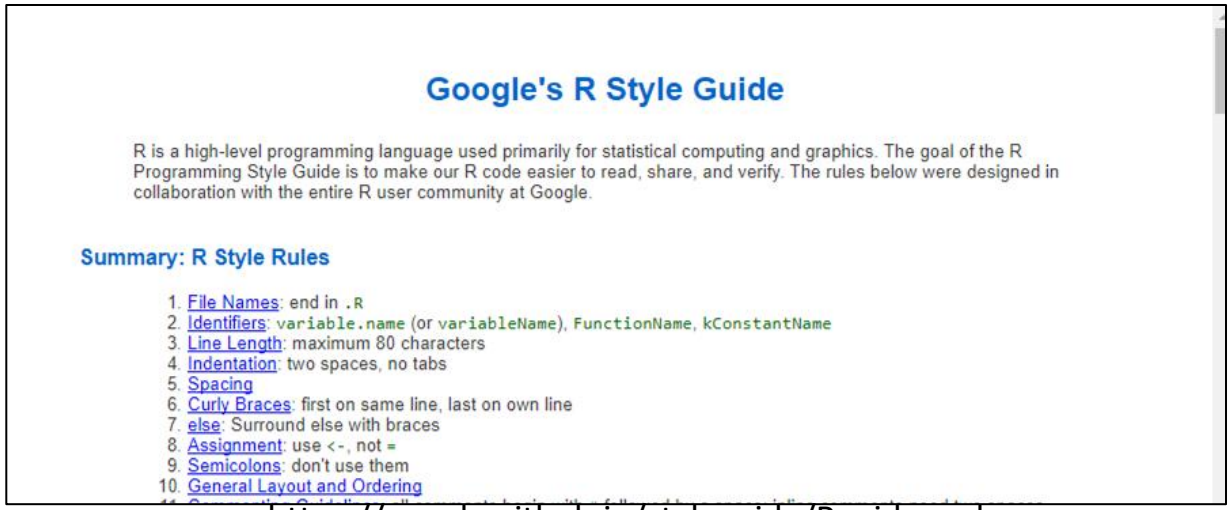
Arithmetic with R

- In the most basic form, R can be used as a simple calculator.
- Using following arithmetic operators:
  - Addition: +
  - Subtraction: -
  - Multiplication: *
  - Division: /
  - Exponentiation: ^
  - Modulo: %%



Variable Assignment in R

- In R, variable assignment is done using either “<-” or “=“
- For example: **“x<-4”** is equivalent to **“x=4”**
- However, some style guide such as Google R Style Guide prefers the use of “<-” instead of “=“



Google's R Style Guide

R is a high-level programming language used primarily for statistical computing and graphics. The goal of the R Programming Style Guide is to make our R code easier to read, share, and verify. The rules below were designed in collaboration with the entire R user community at Google.

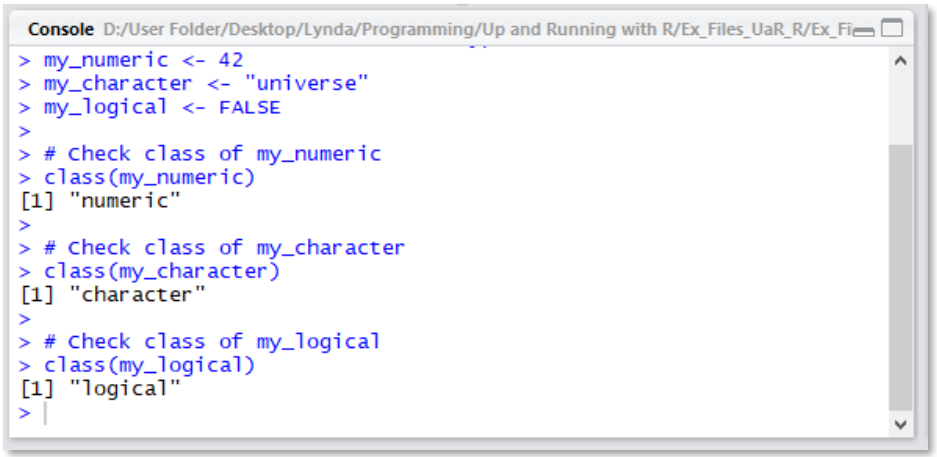
Summary: R Style Rules

1. File Names: end in .R
2. Identifiers: variable.name (or variableName), FunctionName, kConstantName
3. Line Length: maximum 80 characters
4. Indentation: two spaces, no tabs
5. Spacing
6. Curly Braces: first on same line, last on own line
7. else: Surround else with braces
8. Assignment: use <-, not =
9. Semicolons: don't use them
10. General Layout and Ordering

https://google.github.io/styleguide/Rguide.xml

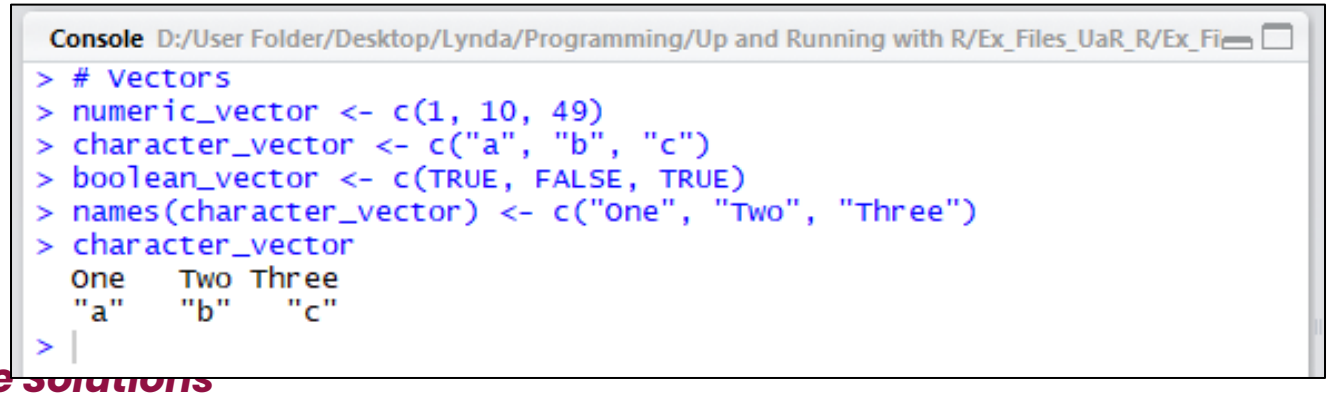
Basic Data Types in R

- Some of the basic data types in R are:
  - **Numeric**: ……1, 10, 15, 1C, 2E,……
  - **Integer**: ……2, 15, 33, 13, 18,……
  - **Logical**: TRUE or FALSE
  - **Characters**: Text or strings
- The data types of a variable can be checked with **class()** function.



Vectors in R

- Vectors are **one-dimension arrays** that can hold numeric, integer, character or logical data.
- In R, vectors are created using function **c()** ,for example:
  - **numeric_vector <- c(1, 10, 49)**
  - **character_vector <- c("a", "b", "c")**
  - **boolean_vector <- c(TRUE, FALSE, TRUE)**
- In R, we can set custom name for the vector. Example:
  - **names(character_vector) <- c("One", "Two", "Three")**



Arithmetic of Vectors in R

- Vectors can perform **element-wise arithmetic operation**.
- Sum and average of the elements in the vector can be calculated using **sum(*vector*)** and **mean(*vector*)** respectively.
- Vectors can used to do comparison, for example:
  - vector = c(1,2,3,4,5)
    vector > 3
    Output:
    FALSE FALSE FALSE TRUE TRUE
- Element in specific position in the vector can be selected using **vector[*position*]**.
  - Bool_value = c(TRUE, FALSE, TRUE, FALSE)
    Vector = c(2, 3, 5, 6)
    Vector[Bool_value]
    Output:
    2 5

Factors in R

- Factors are **categorical variables** that are useful in summary statistics, plots and regressions.
- To create factors in R, use **factor()** function.
- The factor levels can be changed using the **levels()** function.

```
my_vector <- c("L", "S", "L", "M", "M")

# Option 1
my_factor <- factor(my_vector)
levels(my_factor) <- c("Large", "Medium", "Small")

# Option 2
my_factor <- factor(my_vector,
                    levels = c("S", "M", "L"),
                    labels = c("Small", "Medium", "Large"))
```

Matrices in R

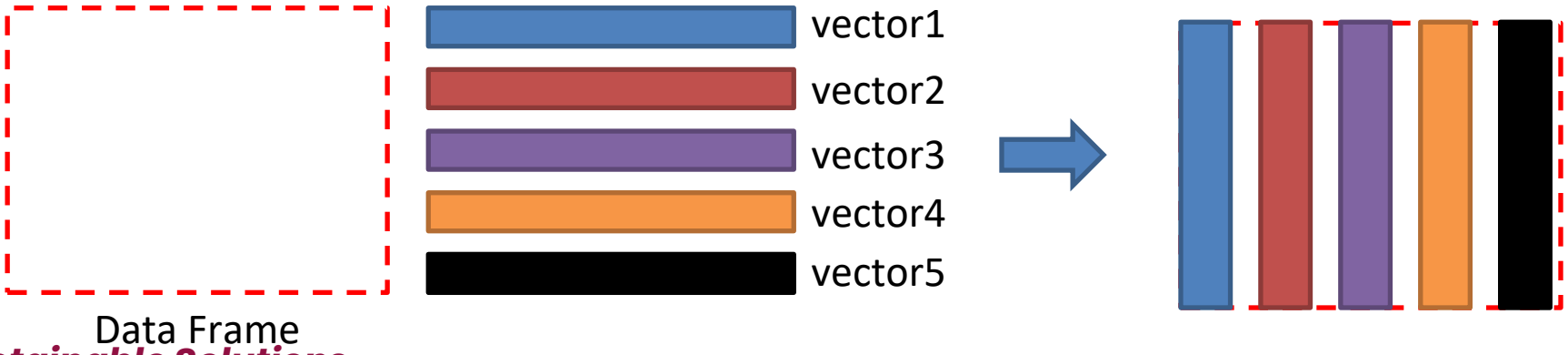
- In R, a matrix is a **collection of elements** of the same data type (numeric, character, or logical) arranged into a fixed number of **rows** and **columns**.
- Since we are only working with rows and columns, a matrix is called **two-dimensional**.
- You can construct a matrix in R with the **matrix()** function. Consider the following example:
  - **matrix(1:9, byrow = TRUE, nrow = 3)**
- The row and column of the matrix can be named using **rownames()** and **colnames()** respectively.
  - Example: **colnames(matrix) <- names**

Operations in Matrices

- We can calculate the column sum and row sum using **colSums()** and **rowSums()** respectively.
  - Example: `colSums(matrix)`
- Additional data can be bind into the matrix using **cbind()** and **rbind()**.
  - `cbind(matrix_A, matrix_B)`: add matrix_B into the "right" side of matrix_A
  - `rbind(matrix_A, matrix_B)`: add matrix_B into the "bottom" side of matrix_A
- Element in specific position in the matrix can be selected using `matrix[row_position, col_position]`.

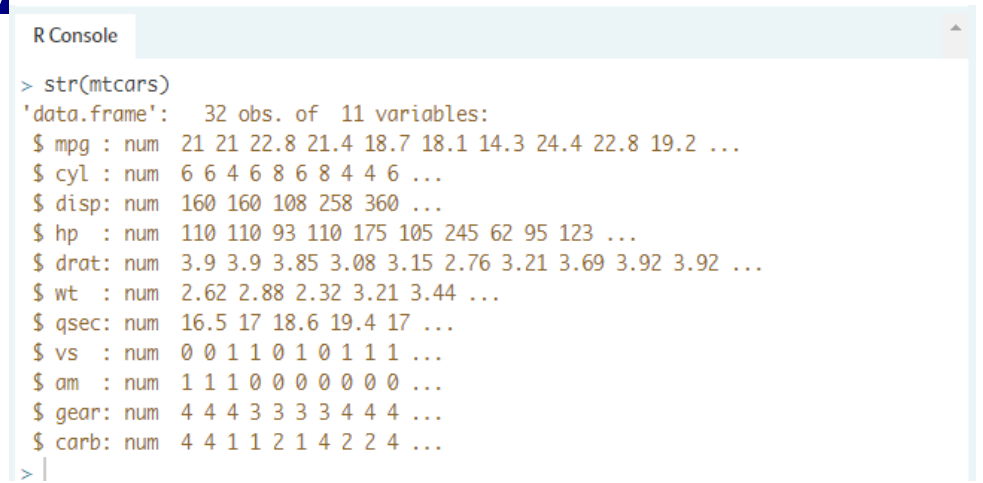
Creating a Data Frame in R

- In R, data in matrix should consists of same data type. However, if the data consists of **different data types**, a **data frame** is more suitable for this case.
- Data frame can be constructed with the **data.frame()** function.
  - Example: data.frame(*vector1, vector2, vector3, vector4, vector 5*)
- Different vectors will be passed as argument and they will become the **different columns of your data frame**.
- Because every column has the same length, the vectors you pass must also have the **same length**.



Data Frame in R

- When dealing with huge data frame, we can use **head(data frame)** or **tail(data frame)** to show only the first few observations or last few observations of the data frame respectively.
- In order to inspect the structure of the data frame, we can use the function **str()**



Accessing Data Frame

- Data frame can be access via **dataframe[row position, column position], dataframe[row position, column name]**
- Column of the data frame also can be assess by using "$"
  - For example: **dataframe$column1**
- Subset of the data frame can be obtained through function **subset(dataframe, subset=condition)**
- The data frame can be sorted using **order()** function.
  - For example: **order(dataframe$column)**

Characters Data in R

- Character's data in R often use **double quotes**.
- **Concatenation** of strings can be done using **paste()** function. For example: paste(string1, string2, sep="")
- To print/display output on the console window, use **print()** function.

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

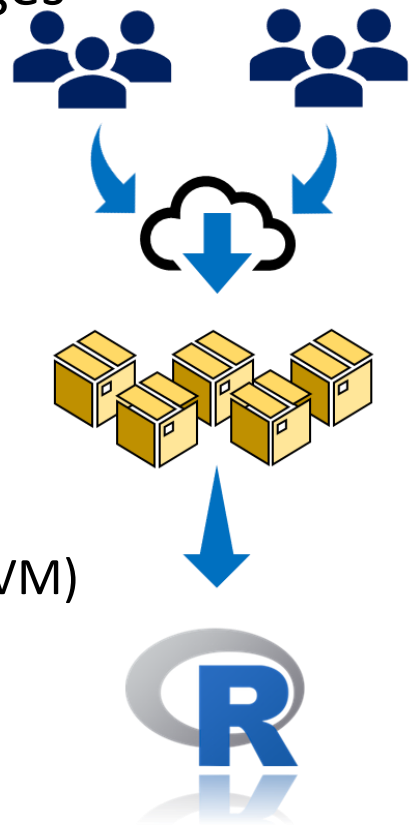


PACKAGES IN R

Innovating Sustainable Solutions

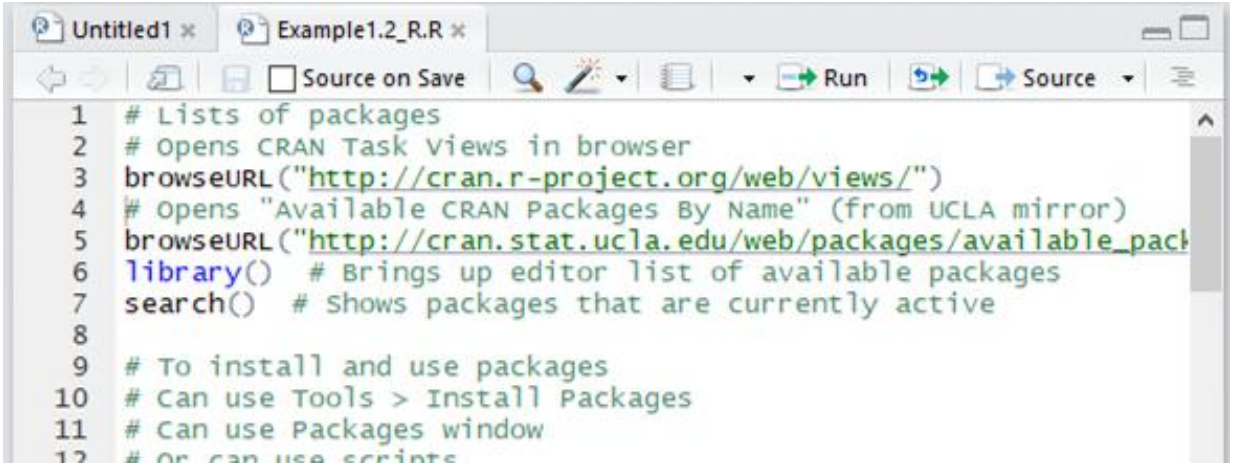
R Packages

- R is highly extensible with widely available packages contributed by community.
- Easy sharing of scripts.
- Comprehensive R Archive Network (CRAN)
- Currently lists over 10000 free packages for R
- Example of packages such as:
  - ggplot2 – for advanced visualization of data
  - e1071 – standard library for support vector machine (SVM)
- To manage R packages, there are two ways:
  - via R Console
  - via RStudio



List available R Packages

- Line 3 code will open up the URL in web browser which shows a large list of categories of packages that are available.
- Line 5 code will shows a long list of packages by name.
- To show the installed packages in the machine, use **library()**
- To show the current active packages, use **search()**



Install R Packages

- There are several ways to **install** R packages in RStudio.
  - Method 1: Using function install.packages()
    - For example: install.packages("package_name")
  - Method 2: Install package through RStudio GUI (online/manual)
    - Can be accessed through:
      - Package pane at the bottom right and click install.
      - Tools menu > Install Packages
- To **load** the installed packages, use:
  - library("*package_name*")
  - require("*package_name*")
- To bring up the **documentation** of the package:
  - library(help="*package_name*")

Update, Detach and Delete R Packages

- To update packages:
  - Use function: **update.packages()**
  - Or through Tools > Checks for Updates
- To unload/detach an active package:
  - Use function: **detach("package:*package_name*", unload=TRUE)**
  - Untick the function that want to detach at Package pane.
- To delete an installed package:
  - Use function: **remove.packages("*package_name*", lib="~/R/win-library/3.4")**
  - Click the "x" at the right of the list of installed package in the Package pane.

UTM
UNIVERSITI TEKNOLOGI MALAYSIA



SEARCH HELP IN R

Innovating Sustainable Solutions

Vignettes in R

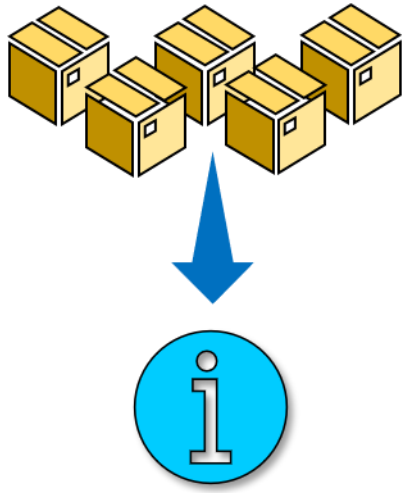
- Most of the R packages have a list of vignettes, which consist of the examples and detail description regarding the package.
- To bring up the list of vignettes of the package:

  vignette(package = "*package_name*")
- To show an interactive list of vignettes of the package in HTML through browser:

  browseVignettes(package = "*package_name*")
- To show all vignettes for all available packages:

  vignette()

  browseVignettes()



UTM
UNIVERSITI TEKNOLOGI MALAYSIA



FUNCTIONS IN R

Innovating Sustainable Solutions

Define a function

- To create a “trigger” for a series of action in a more manageable way.
- Note: the R script must be loaded first before the function can be called in the R Console.

Script.R

```
myFunctName <- function() {
        print("Hello")
}

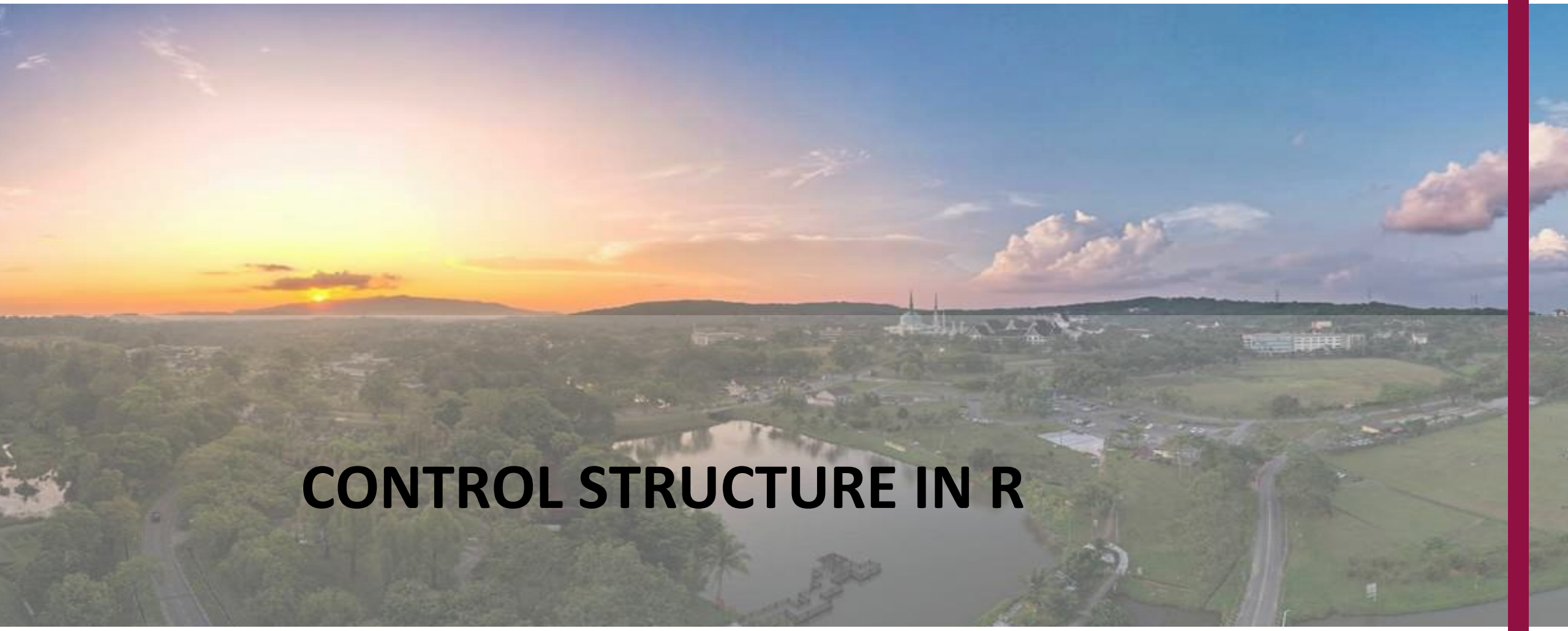
Function name                    Parameters
myFunct2Name <- function(param1, param2)
{
        print(param1+param2)
}

Function name
myFunct3Name <- function(param1) {
        myFunction2Name(param1, 10)
        print(param1)
}
```

R Console

```
> myFunctName()
[1] Hello
> myFunct2Name(15, 4)
[1] 19
> myFunct3Name(25)
[1] 35
[1] 25
```

UTM
UNIVERSITI TEKNOLOGI MALAYSIA



CONTROL STRUCTURE IN R

Innovating Sustainable Solutions

Control Structures in R

- If……Else in R

```
if(condition) {
        #statement
}
```

- For loop in R

```
for(x in 1: 10) {
        print(x)
}
```

- While loop in R

```
x <- 1;
while(x<11) {
        print(x)
        x = x+1;
}
```

UTM
UNIVERSITI TEKNOLOGI MALAYSIA



IMPORT/EXPORT IN R

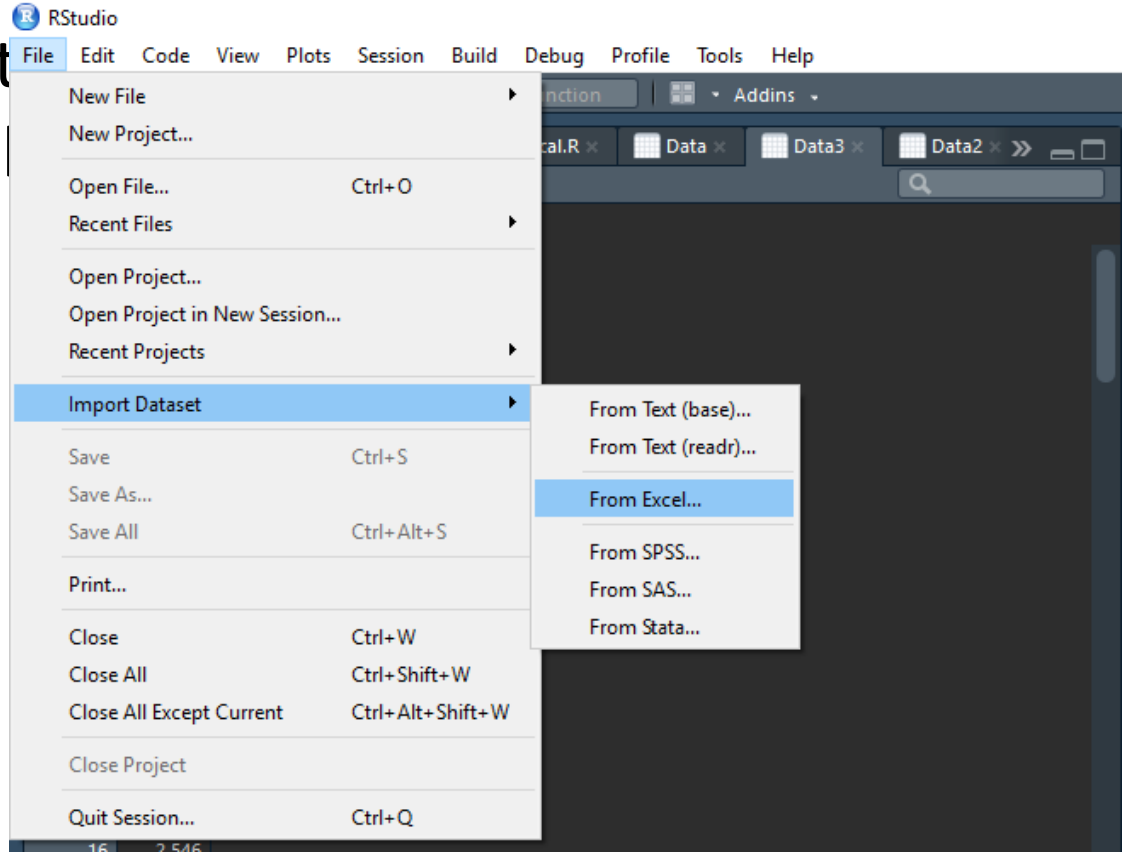
Innovating Sustainable Solutions

Save/Load and Import/Export

- In R, data can be save into R binary data (.Rdata extension), which only readable by R.
- This can be done by **save()** function.
  - For example: save(*variable*, file="*path/filename.Rdata*")
  - The data saved in Rdata format can be loaded into R using **load()**
- In R, data also can be exported to a Excel-readable format (.csv) using **write.csv()**
  - For example: write.csv(*dataframe, "path/filename.csv"*)
  - The CSV file can be imported into R using **read.csv()**

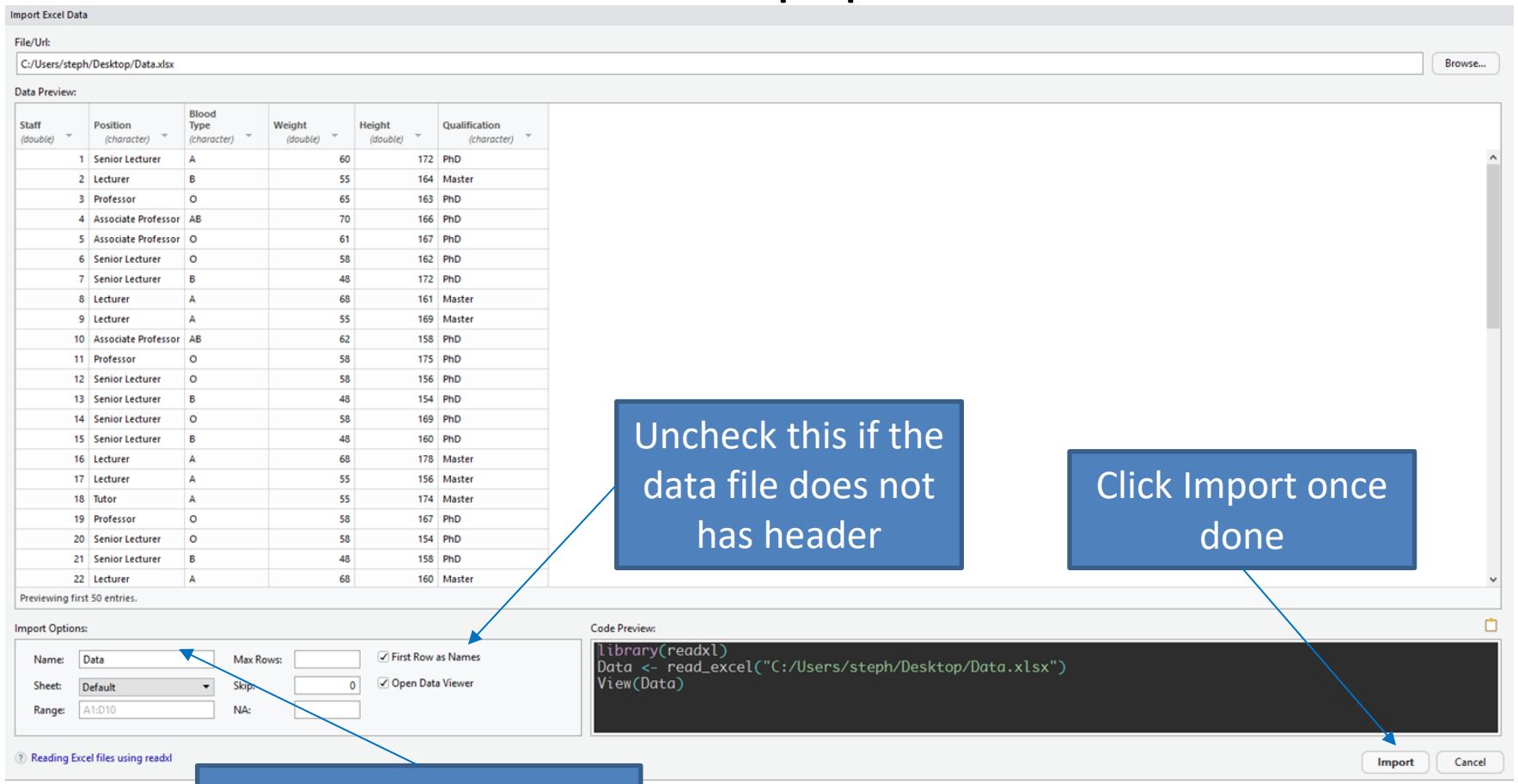
Import from Excel

- In R, Excel file can be imported by navigating to the option pane: File Import Dataset From Excel
- If first
  it wi
  download
  install a package.

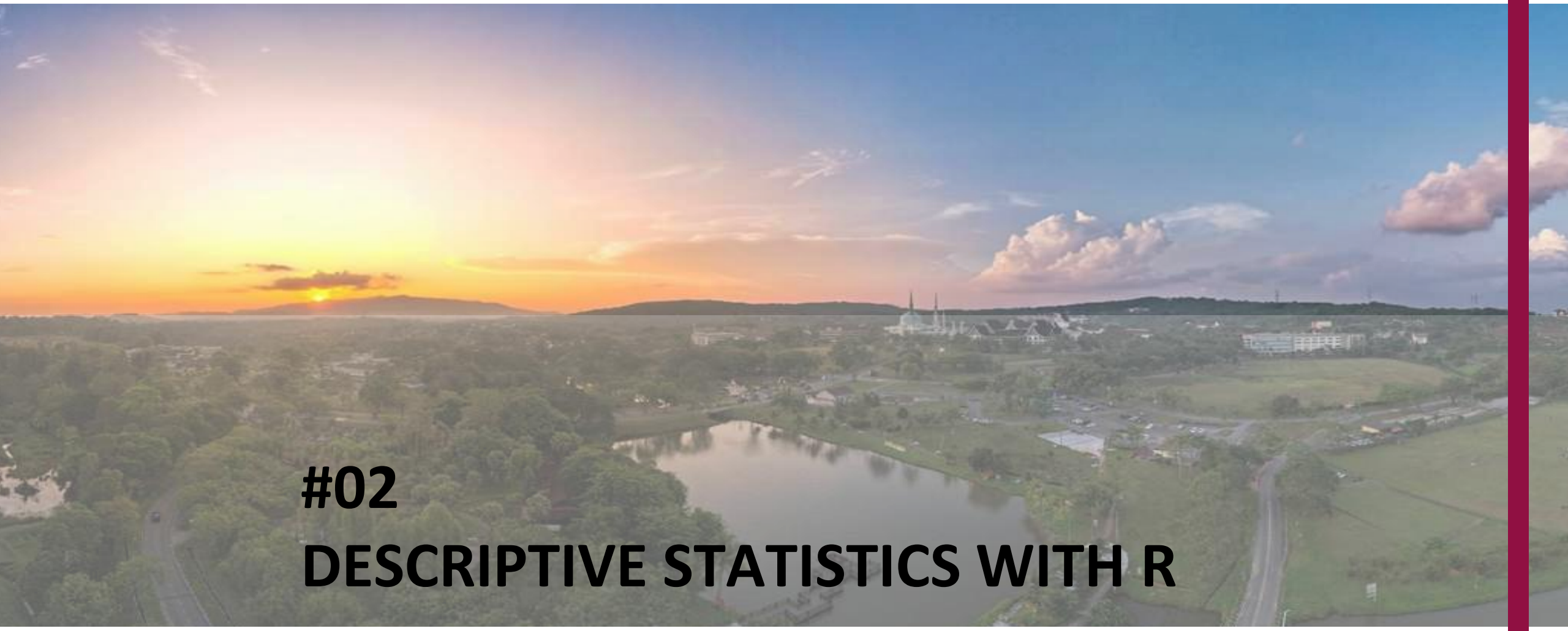


Import from Excel

- File selection window will pop out as follow:



UTM
UNIVERSITI TEKNOLOGI MALAYSIA



#02
DESCRIPTIVE STATISTICS WITH R

Innovating Sustainable Solutions

Presenting Categorical Data

Frequency Distribution

- To organize the data using table() function in R
- Example (refer #lectureTWO, slide 7):

	A	B	C	D	E	F
1	Staff	Position	Blood Type	Weight	Height	Qualification
2	1	Senior Lecturer	A	60	172	PhD
3	2	Lecturer	B	55	164	Master
4	3	Professor	O	65	163	PhD
5	4	Associate Professor	AB	70	166	PhD
6	5	Associate Professor	O	61	167	PhD
7	6	Senior Lecturer	O	58	162	PhD
8	7	Senior Lecturer	B	48	172	PhD
9	8	Lecturer	A	68	161	Master
10	9	Lecturer	A	55	169	Master
11	10	Associate Professor	AB	62	158	PhD
12	11	Professor	O	58	175	PhD
13	12	Senior Lecturer	O	58	156	PhD
14	13	Senior Lecturer	B	48	154	PhD
15	14	Senior Lecturer	O	58	169	PhD
16	15	Senior Lecturer	B	48	160	PhD
17	16	Lecturer	A	68	178	Master
18	17	Lecturer	A	55	156	Master
19	18	Tutor	A	55	174	Master
20	19	Professor	O	58	167	PhD
21	20	Senior Lecturer	O	58	154	PhD
22	21	Senior Lecturer	B	48	158	PhD

Academic staff info: 140 staffs

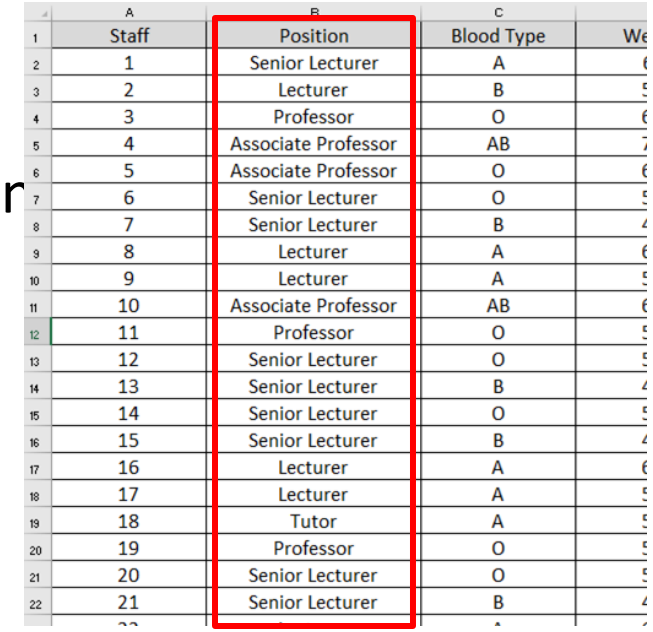
Steps to analyze the data:

1. Import the data into R
2. Extract frequency distribution for desired information
3. Plot the frequency distribution in a bar plot

Presenting Categorical Data

Frequency Distribution

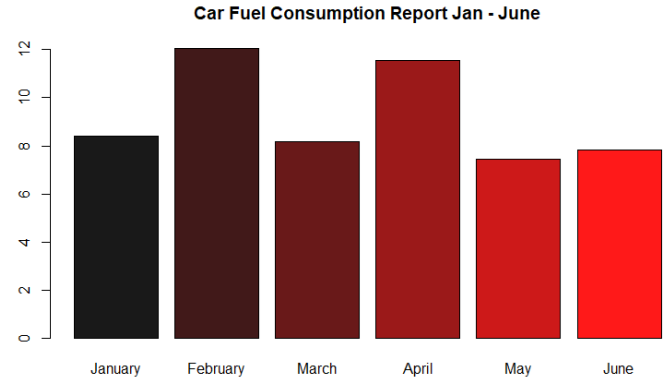
- **STEP 1:** At the menu bar of Rstudio, go to: File ⍰ Import Data ⍰ From Excel select the data file and import into the R environment.
- **STEP 2:** Extract the frequency distribution information using following command:
  staffPosition.freq <- table(Data[,2])
- **STEP 3:** Plot the frequency distribution using bar plot
  barplot(staffPosition.freq)

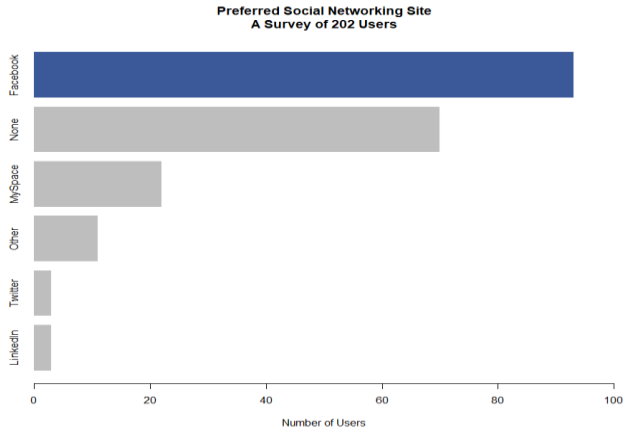


	A	B	C	
1	Staff	Position	Blood Type	We
2	1	Senior Lecturer	A	
3	2	Lecturer	B	
4	3	Professor	O	
5	4	Associate Professor	AB	
6	5	Associate Professor	O	
7	6	Senior Lecturer	O	
8	7	Senior Lecturer	B	
9	8	Lecturer	A	
10	9	Lecturer	A	
11	10	Associate Professor	AB	
12	11	Professor	O	
13	12	Senior Lecturer	O	
14	13	Senior Lecturer	B	
15	14	Senior Lecturer	O	
16	15	Senior Lecturer	B	
17	16	Lecturer	A	
18	17	Lecturer	A	
19	18	Tutor	A	
20	19	Professor	O	
21	20	Senior Lecturer	O	
22	21	Senior Lecturer	B	

Bar Plot in R

- In R, bar chart / barplot is plotted using **barplot()** function.
- **barplot()** is a built-in function
- Example: **bar(** *variable*,
  col=*color*,
  border=*NA*,
  xlim=*range*,
  main=*title*,
  xlab=*x-axis*,
  ylab=*y-axis name***)**

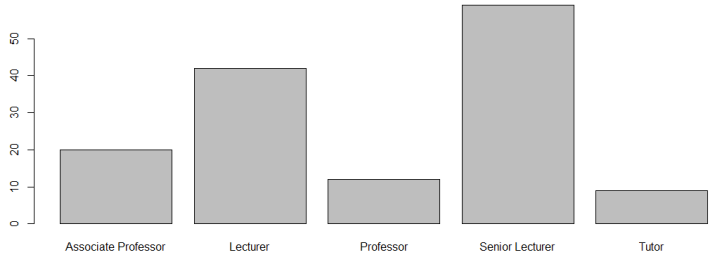




Presenting Data using barplot()

- To plot the academic staff data into a bar plot, use following command:

```
barplot(staffPosition.freq)
```



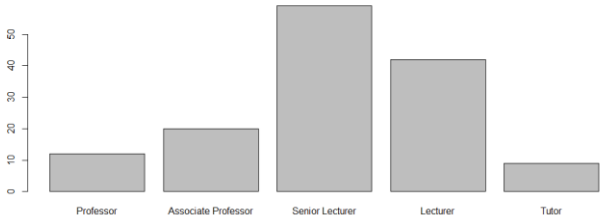
- What if we want to reorder it?
- Using following command in R we can reorder the data:

```
staffPosition = staffPosition.freq [c(3,1,4,2,5)]
```

- Then replot again for the new ordered data:

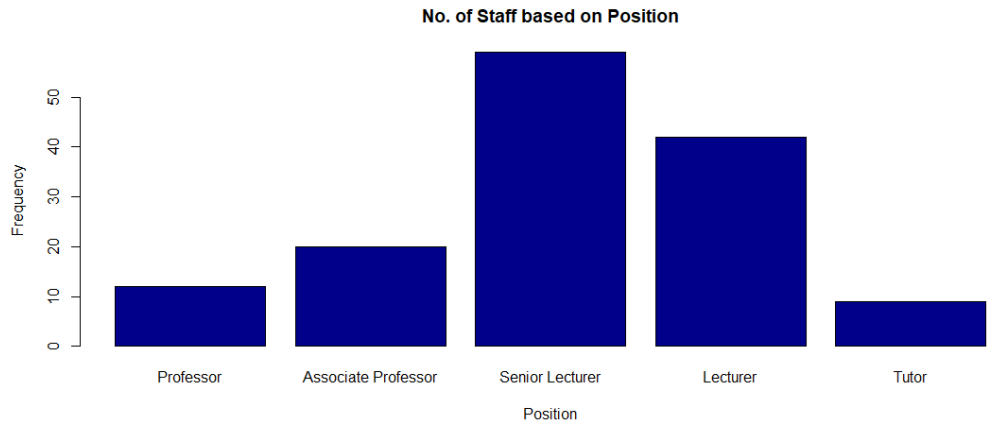
```
barplot(staffPosition)
```

Reorder based on the index



Settings in barplot()

- Settings of bar plot available in R include:
  - Color
  - Bar orientation
  - Range
  - Title, labels
- For example:




```
barplot(staffPosition, col="darkblue", xlab="Position",
ylab="Frequency", main="No. of Staff based on Position")
```

Presenting Data using pie()

- Example: let say we have a set of data of number of students for 2017/2018 Intake as follow:
- Let say we import the data and save it into a variable name **Data2**
- After import the data in R, we can add the column names using:

```
colnames(Data2) <- c("X1", "X2")
```

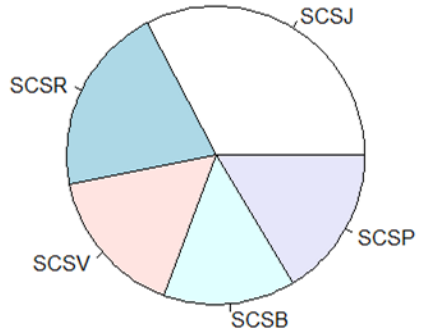
- Then, a pie chart can be plotted using following command:

```
pie(Data2$X2, Data2$X1)
```

Data to be plotted

Labels

Course	No. of students
SCSJ	80
SCSR	50
SCSV	40
SCSB	35
SCSP	40



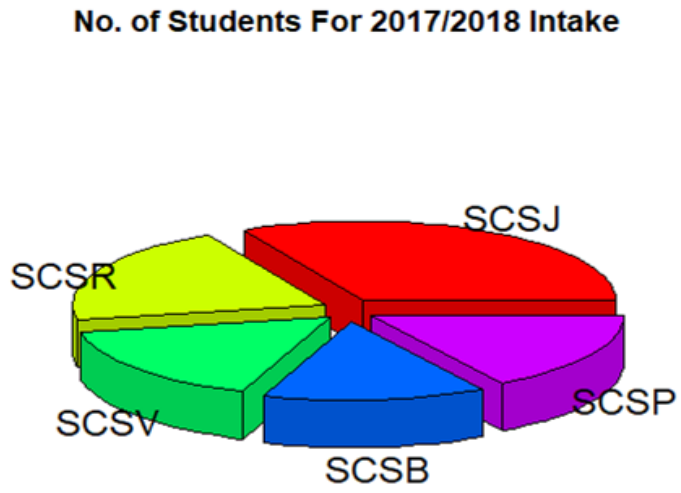
More with pie chart in R

- 3D pie chart is available with the use of package ***plotrix***
  install.packages("plotrix")
  library("plotrix")
- After install the package, use the function **pie3D()**
- Example using the student data:

```
pie3D(Data2$X2, labels=Data2$X1,
      main="No. of Students For 2017/2018 Intake",
      explode=0.1)
```

The function of **pie3D** required package named "**plotrix**" to work. Please install the package before you try this.

"explode" parameter for the separation between each slice



Relative Frequency Distribution

- Refer back to the academic staff data in Slide 3.
- Let say we already have the frequency distribution in the variable ***staffPosition.freq***
- We can calculate the relative frequency distribution simply with following command

Position	Number of Staff (frequency)
Professor	12
Associate Professor	20
Senior Lecturer	59
Lecturer	40
Tutor	9
Total	140

```
staffPosition.rfd <- staffPosition.freq/140
```

- In R, arithmetic executed for a list variable will apply to each of the element.

```
> staffPosition.rfd <- staffPosition.freq/140
> staffPosition.rfd

        Professor Associate Professor     Senior Lecturer            Lecturer               Tutor
       0.08571429          0.14285714          0.42142857          0.28571429          0.06428571
> |
```

- Then plot the relative frequency distribution into a bar plot

Presenting Data using stem and leaf

- Let say we have a list of numbers in a variable named **num**:

  12, 13, 21, 27, 33, 34, 35, 37, 40, 40, 41

- In R in order to draw a stem-and-leaf plot, we can use the stem() function.

  stem(num)

```
> num <- c(12, 13, 21, 27, 33, 34, 35, 37, 40, 40, 41)
> stem(num)

  The decimal point is 1 digit(s) to the right of the |

  1 | 23
  2 | 17
  3 | 3457
  4 | 001
```

x: Please specify the data on which you want to draw the Stem and Leaf Plot. Here, you have to use the numeric vector, or a list containing numeric vector.

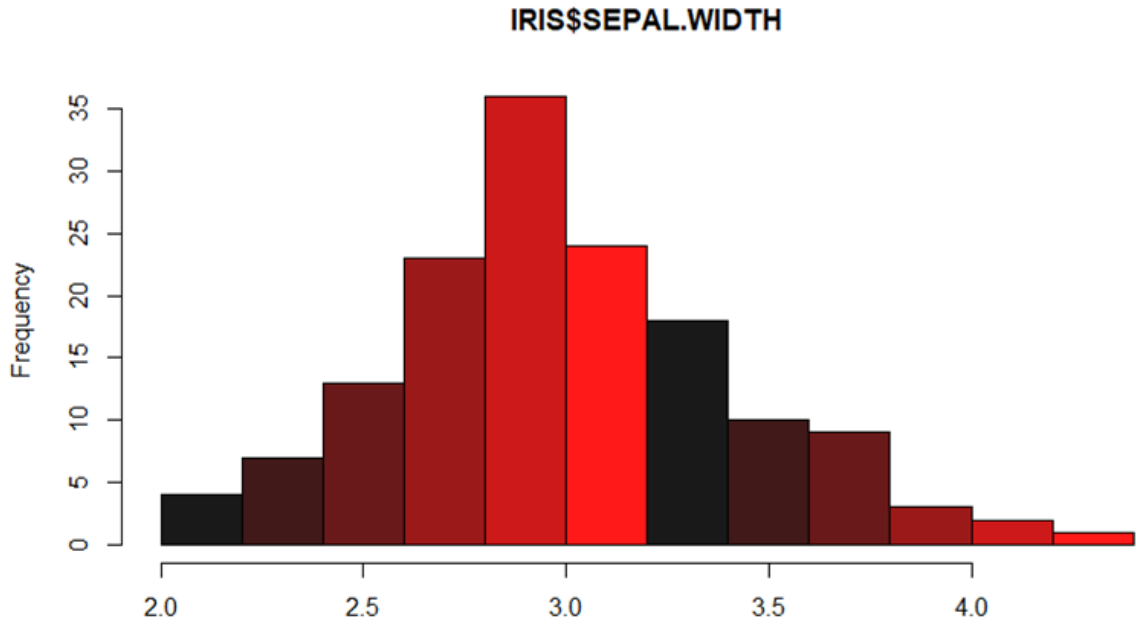
scale: Please specify the scale you want to use for your plot.

width: It is optional but you can use this to specify the desired width of a plot. By default it is 80

atom: It is a tolerance

Histogram in R

- In R, histogram is plotted using **hist()** function.
  - Example: **hist(***variable,* **col=***color,* **border=***NA*, **xlim=***range*, **main=***title*, **xlab=***x-axis,* **ylab***=y-axis name**)***



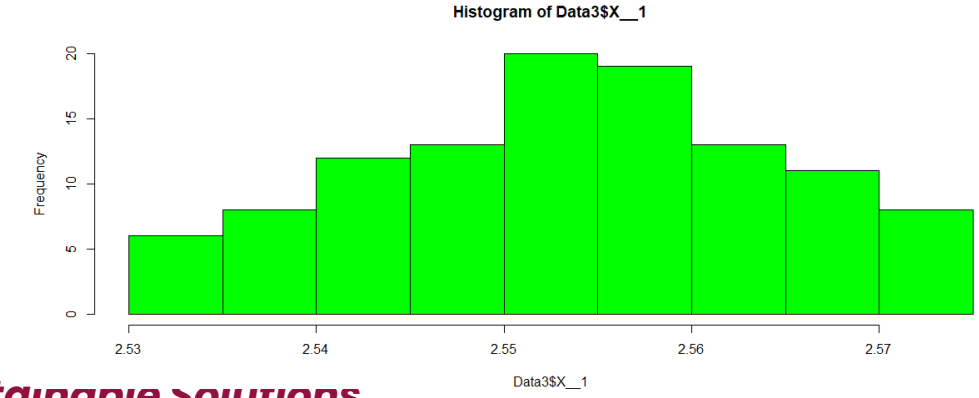
Presenting data in histogram

- Let say we have a group of data and imported into R in a variable named **Data3**
- Add column name using: `colnames(Data3) <- c("X1")`
- To plot a histogram out of the data, we can use following:

```
hist(Data3$X1, col="green")
```

- We can save the output by assigning the command above into a variable.

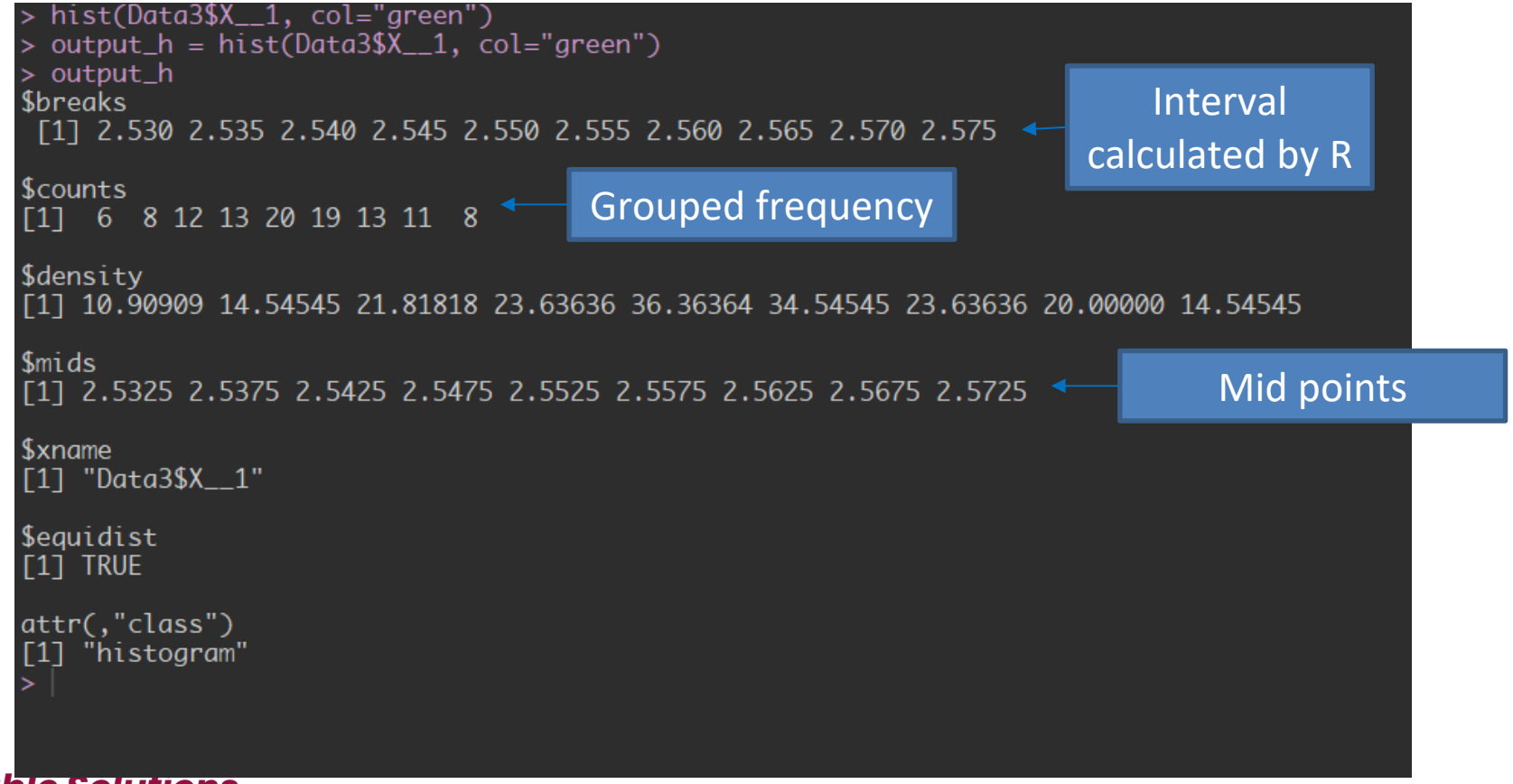
```
output_h = hist(Data3$X1, col="green");
```



2.559	2.556	2.566	2.546	2.561	2.570	2.546
2.565	2.543	2.538	2.560	2.560	2.545	2.551
2.568	2.546	2.555	2.551	2.554	2.574	2.568
2.572	2.550	2.556	2.551	2.561	2.560	2.564
2.567	2.560	2.551	2.562	2.542	2.549	2.561
2.556	2.550	2.561	2.558	2.556	2.559	2.557
2.532	2.575	2.551	2.550	2.559	2.565	2.552
2.560	2.534	2.547	2.569	2.559	2.549	2.544
2.550	2.552	2.536	2.570	2.564	2.553	2.558
2.538	2.564	2.552	2.543	2.562	2.571	2.553
2.539	2.569	2.552	2.536	2.537	2.532	2.552
2.575 (h)	2.545	2.551	2.547	2.537	2.547	2.533
2.538	2.571	2.545	2.545	2.556	2.543	2.551
2.569	2.559	2.534	2.561	2.567	2.572	2.558
2.542	2.574	2.570	2.542	2.552	2.551	2.553
2.546	2.531 (l)	2.563	2.554	2.544		

Presenting data in histogram

- This variable holds the information calculated by R during the construction of the histogram.

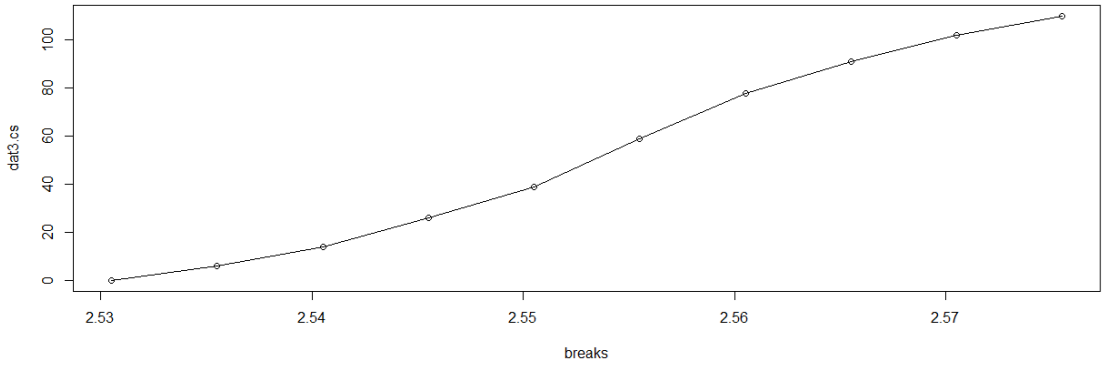


Frequency distribution of grouped data

- Consider the data used in histogram plotting.
- We can group the data based on desired breaks.
- Generally, follow steps below
  1. Extract the data (if it is in dataframe) into a list or vector
  2. Define the breaks (a.k.a interval)
  3. Group the data based on the breaks
  4. Calculated cumulative frequency
  5. Plot the result.

2.559	2.556	2.566	2.546	2.561	2.570	2.546
2.565	2.543	2.538	2.560	2.560	2.545	2.551
2.568	2.546	2.555	2.551	2.554	2.574	2.568
2.572	2.550	2.556	2.551	2.561	2.560	2.564
2.567	2.560	2.551	2.562	2.542	2.549	2.561
2.556	2.550	2.561	2.558	2.556	2.559	2.557
2.532	2.575	2.551	2.550	2.559	2.565	2.552
2.560	2.534	2.547	2.569	2.559	2.549	2.544
2.550	2.552	2.536	2.570	2.564	2.553	2.558
2.538	2.564	2.552	2.543	2.562	2.571	2.553
2.539	2.569	2.552	2.536	2.537	2.532	2.552
2.575 (h)	2.545	2.551	2.547	2.537	2.547	2.533
2.538	2.571	2.545	2.545	2.556	2.543	2.551
2.569	2.559	2.534	2.561	2.567	2.572	2.558
2.542	2.574	2.570	2.542	2.552	2.551	2.553
2.546	2.531 (l)	2.563	2.554	2.544		





Frequency distribution of grouped data

- STEP 1: Extract the data (if it is in dataframe) into a list or vector

```
Dat3 <- Data3$X1
```

Extract the info and assign into Dat3 which is a vector

- STEP 2: Define the breaks (a.k.a interval)

```
breaks <- seq(2.5305, 2.5375, 0.005)
```

(low limit, up limit, interval)

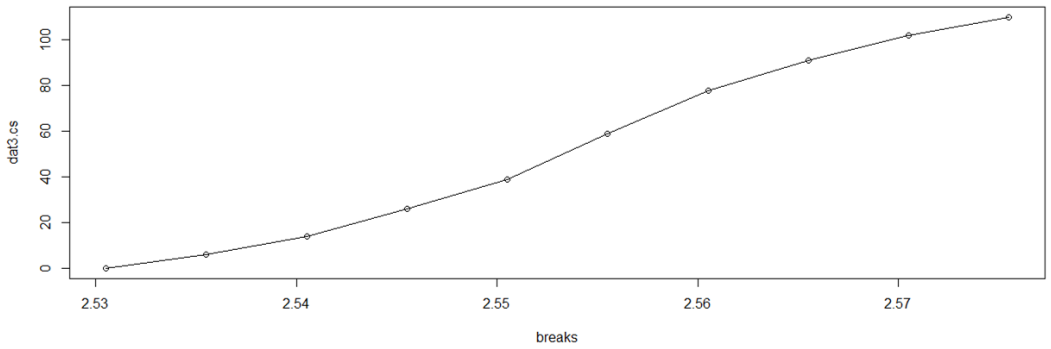
- STEP 3: Group the data based on the breaks

```
Dat3.cut <- cut(Dat3, breaks)
Dat3.freq <- table(Dat3.cut)
```

Categorize each data based on the breaks

Group the similar breaks and get the frequency

2.559	2.556	2.566	2.546	2.561	2.570	2.546
2.565	2.543	2.538	2.560	2.560	2.545	2.551
2.568	2.546	2.555	2.551	2.554	2.574	2.568
2.572	2.550	2.556	2.551	2.561	2.560	2.564
2.567	2.560	2.551	2.562	2.542	2.549	2.561
2.556	2.550	2.561	2.558	2.556	2.559	2.557
2.532	2.575	2.551	2.550	2.559	2.565	2.552
2.560	2.534	2.547	2.569	2.559	2.549	2.544
2.550	2.552	2.536	2.570	2.564	2.553	2.558
2.538	2.564	2.552	2.543	2.562	2.571	2.553
2.539	2.569	2.552	2.536	2.537	2.532	2.552
2.575 (h)	2.545	2.551	2.547	2.537	2.547	2.533
2.538	2.571	2.545	2.545	2.556	2.543	2.551
2.569	2.559	2.534	2.561	2.567	2.572	2.558
2.542	2.574	2.570	2.542	2.552	2.551	2.553
2.546	2.531 (l)	2.563	2.554	2.544		



Frequency distribution of grouped data

- STEP 4: Calculated cumulative frequency using **cumsum()**

```
Dat3.cs <- c(0, cumsum(Dat3.freq))
```

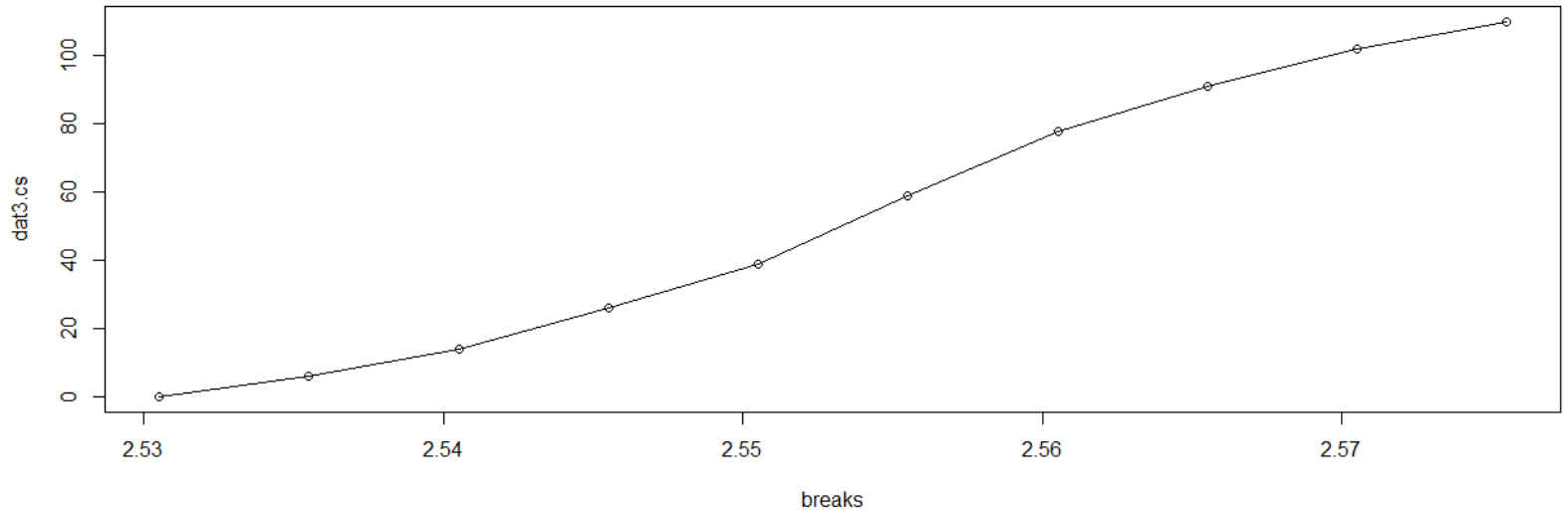
Calculate cumulative freq

- STEP 5: Plot the results using **plot()** function

```
plot(breaks, Dat3.cs)
lines(breaks, Dat3.cs)
```

Plot a graph

Link the dots with line



Presenting Data in boxplot()

- Let say we have a set of data:

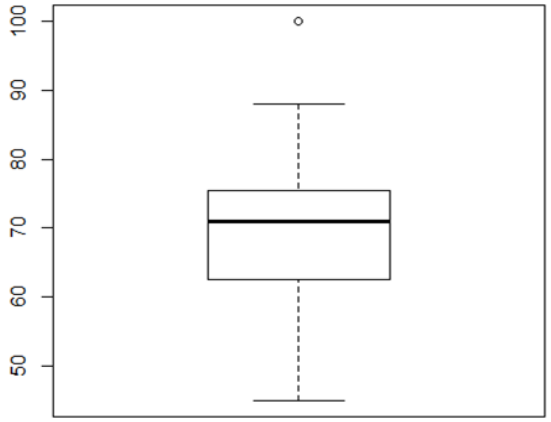
45 48 50 54 57 60 60 62 63 63 64 65 67 68 69
71 71 72 72 74 74 75 75 76 80 83 84 88 100 100
72 100

- First we have to make sure the data we have is in data frame in R. Let say we save this in a variable named **num2**
- Then to plot a boxplot, simply use following command:

```
boxplot(num2)
```

- We can also save the output of the boxplot function into a variable, let say **output**

```
output = boxplot(num2)
```



More in boxplot in R

- By assigning the output of a boxplot into a variable, information calculated by R can be revealed.

A list is returned which has

stats - having the position of the upper/lower extremes of the whiskers and box along with the median

n - the number of observation the boxplot is drawn with (notice that NA‘s are not taken into account)

conf - upper/lower extremes of the notch, out-value of the outliers

group - a vector of the same length as out whose elements indicate to which group the outlier belongs and

names - a vector of names for the groups.

```
> output <- boxplot(num2)
> output
$stats
     [,1]
[1,] 45.0
[2,] 62.5
[3,] 71.0
[4,] 75.5
[5,] 88.0

$n
[1] 31

$conf
         [,1]
[1,] 67.31091
[2,] 74.68909

$out
[1] 100 100 100

$group
[1] 1 1 1

$names
[1] "bdata"

>
```

Descriptive Statistics (Mean)

- Mean can be calculated using **mean()** function.
- For simple data such as a sequence of numbers. Let say as following:

```
num3 = c(20, 30, 40, 50, 80, 100)
```

- The mean can be calculated simply with following command:

```
mean(num3)
```

```
> num3 = c(20, 30, 40, 50, 80, 100)
>
> mean(num3)
[1] 53.33333
>
```

Descriptive Statistics (Median)

- Median can be calculated using **median()** function.
- Refer back to the same data:

```
num3 = c(20, 30, 40, 50, 80, 100)
```

- The median can be calculated simply with following command:

```
median(num3)
```

```
> num3 = c(20, 30, 40, 50, 80, 100)
>
> median(num3)
[1] 45
>
```

Descriptive Statistics (Mode)

- R **does not** have a standard in-built function to calculate mode. So we create a user function to calculate mode of a data set in R.
- Assume we have a list of numbers as follows:

```
num4 = c(20, 30, 40, 50, 20, 80, 100)
```

- Mode can be calculated using following steps:
- STEP 1: get the available unique number using **unique()**

```
unique.value <- unique(num4)
```

- STEP 2: find occurrence of unique number using **match()**

```
unique.match <- match(num4, unique.value)
```

Descriptive Statistics (Mode)

- STEP 3: find the frequency of each unique number using **tabulate()**

```
unique.freq <- tabulate(unique.match)
```

- STEP 4: find the index which has the highest frequency using **which.max()**

```
unique.max <- which.max(unique.freq)
```

- STEP 5: get the value of the mode by using the index identified in STEP 4

```
num4_mode <- unique.value[unique.max]
```

```
> unique.value <- unique(num4)
> unique.value
[1]  20  30  40  50  80 100
> unique.match <- match(num4, unique.value)
> unique.match
[1] 1 1 2 3 4 5 6
> unique.freq <- tabulate(unique.match)
> unique.freq
[1] 2 1 1 1 1 1
> unique.max <- which.max(unique.freq)
> unique.max
[1] 1
> num4_mode <- unique.value[unique.max]
> num4_mode
[1] 20
>
```

Descriptive Statistics (Quartile)

- In R, information for quartile can be obtained using quantile() function.
- Let say we have a list of data as follows:

```
num5 = c(12, 4, 6, 11, 9, 15, 20, 18, 25, 30)
```

- The quartile information can be obtained via following:

```
quantile(num5, type=2)
```

- There are several types of setting for the function that will generate different results

```
> quantile(num5, type=2)
  0%  25%  50%  75% 100%
 4.0  9.0 13.5 20.0 30.0
> |
```

  - Source: https://tolstoy.newcastle.edu.au/R/e17/help/att-1067/Quartiles_in_R.pdf

Descriptive Statistics (variance / std dev)

- In R, variance can be calculated using the **var()** function.
- Using the same data as previous slide, variance can be obtained using the following command:

```
num5_variance <- var(num5)
```

- While for standard deviation, can be obtained using **sd()** in R
- For example:

```
num5_sd <- sd(num5)
```

```
> num5_variance <- var(num5)
> num5
 [1] 12  4  6 11  9 15 20 18 25 30
> num5_variance
[1] 69.11111
>
```

```
> num5_sd <- sd(num5)
> num5
 [1] 12  4  6 11  9 15 20 18 25 30
> num5_sd
[1] 8.313309
>
```

Skewness in R

- In R, skewness can be measured using **skewness()** function in the package named “moments”
- Let say we have a set of data as follows:

```
time <- c(19.09, 19.55, 17.89, 17.73, 25.15, 27.27, 25.24, 21.05, 21.65, 20.92, 22.61, 15.71, 22.04, 22.60, 24.25)
```

- The skewness of the data can be measured using following command:

```
skewness(time)
```

This required package named “**moments**” to work. Please install the package before you try this.

```
> time <- c(19.09, 19.55, 17.89, 17.73, 25.15, 27.27, 25.24,
21.05,  21.65, 20.92, 22.61, 15.71, 22.04, 22.60, 24.25)
>
> skewness(time)
[1] -0.01565162
> |
```

Kurtosis in R

- In R, kurtosis can be measured using **kurtosis()** function in the package named “moments” as same as previous.
- Let say we use the same data as previous slide:

```
time <- c(19.09, 19.55, 17.89, 17.73, 25.15, 27.27, 25.24, 21.05, 21.65, 20.92, 22.61, 15.71, 22.04, 22.60, 24.25)
```

- The kurtosis of the data can be measured using following command:

```
kurtosis(time)
```

This required package named “**moments**” to work. Please install the package before you try this.

```
> time <- c(19.09, 19.55, 17.89, 17.73, 25.15, 27.27, 25.24,
21.05,  21.65, 20.92, 22.61, 15.71, 22.04, 22.60, 24.25)
>
> kurtosis(time)
[1] 2.301051
>
```